Revision history

Revision	Date	Changes
6	19-Aug-2014	- Introduced definition of *<addressees>*. - Introduced distinction between *show* and *send*.
7		- Changed syntax to angle brackets.
8	08-Oct-2015	- Removed angle brackets and replaced it with Python like syntax.

Hippocrates examples

Simple adherence reminder

```
"simple" is a plan:
  30 minutes before breakfast, 30 minutes before dinner:
     show message to patient "Please take your dose now":
       message expires after 30 minutes.
```

More complex adherence reminder with automatic extension of a period

```
"infection prevention" is a plan:

  "antibiotics" for 9 days, can be extended to 14 days:
    30 minutes before breakfast, 30 minutes before dinner:
      show message to patient "Please take your dose now":
        message expires after 30 minutes.

    every second day with begin of breakfast:
      show message to patient "Please take the full dose
          of antibiotics to avoid creating resistant germs".

  after antibiotics:
    ask "How many tablets do you still have?":
      0 ... 2:
        show message "Thank you for taking the full dose of
          antibiotics".
      3 ... 20:
        assess current duration of antibiotics:
          0 days ... 8 days:
            do nothing.
          9 days ... 13 days:
            show message to patient "Please take all of the
              antibiotics to avoid creating resistant germs".
```

```
          extend current antibiotics period by 1 day.
        14 days:
          show message to patient "You still seem to have too
            many tablets remaining. Please see your
            physician for further consultation."
          stop plan.
```

An assessment of the respiratory rate and translation into a meaning according to age

```
#-----------------------------------------------------------------------

"age" is a value:
  valid unit is years.
  valid values are between (0 years ... 150 years).
  confidence of age is:
    0 years ...  90 years:
      confidence of value = 100%.
    91 years ... 100 years:
      confidence of value = 90%.
    101 years ... 150 years:
      confidence of value is 70%.
  question to get age:
    ask "Please provide your age in years".

#-----------------------------------------------------------------------

"pediatric age" ia a value:
  valid unit is months.
  valid values are between (0 months ... 12 months).
  question to get pediatric age:
    ask patient "Please provide the age in months".

#-----------------------------------------------------------------------

"respiratory rate" is a value:
  value unit is (breaths per minute, BsPM):
    1 breaths per minute = 1 BsPM.
    1 BsPM = 1 breaths per minute.
  valid values are between (0 BsPM... 100 BsPM).

  meaning of respiratory rate:
    assess age:
      age is 0 years:
        assess pediatric age:
          pediatric age is between (0 months ... 2 months):
            assess respiratory rate:
              0 BsPM ... 19 BsPM:
                meaning = "critically low".
              20 BsPM... 29 BsPM:
                meaning = "low".
              30 BsPM ... 60 BsPM:
                meaning = "normal".
```

```
            61 BsPM ... 80 BsPM:
              meaning = "high".
            81 BsPM ... 100 BsPM:
              meaning = "critically high".
        pediatric age is between (3 months ... 12 months):
          assess respiratory rate:
            0 BsPM ... 19 BsPM:
              meaning = "critically low".
            20 BsPM ... 29 BsPM:
              meaning = "low".
            24 BsPM ...  30 BsPM:
              meaning = "normal".
            31 BsPM ...  50 BsPM:
              meaning = "high".
            51 BsPM ... 100 BsPM:
              meaning = "critically high".
      age is between (1 year ... 5 years):
        assess respiratory rate:
          0 BsPM ... 14 BsPM:
            meaning = "critically low".
          15 BsPM ... 19 BsPM:
            meaning = "low".
          20 BsPM ... 30 BsPM:
            meaning = "normal".
          31 BsPM ... 40 BsPM:
            meaning = "high".
          41 BsPM ... 100 BsPM:
            meaning = "critically high".
      age is between (6 years ... 12 years):
        assess respiratory rate:
          0 BsPM ... 5 BsPM:
            meaning = "critically low".
          6 BsPM ... 11 BsPM:
            meaning = "low".
          12 BsPM ... 20 BsPM:
            meaning = "normal".
          21 BsPM ...  30 BsPM:
            meaning = "high".
          31 BsPM ... 100 BsPM:
            meaning = "critically high".
      age is between (13 years ... 150 years):
        assess respiratory rate:
          0 BsPM ... 5 BsPM:
            meaning = "critically low".
          6 BsPM ... 11 BsPM:
            meaning = "low".
          12 BsPM ... 16 BsPM:
            meaning = "normal".
          17 BsPM ... 30 BsPM:
            meaning = "high".
          31 BsPM ... 100 BsPM:
            meaning = "critically high".

#-----------------------------------------------------------------------
```

In a script the respiratory rate can be used as follows. There will be no need to make all the decisions that are taken within the definition of respiratory rate.

```
plan:
  ...
  assess respiratory rate:
    critically high; critically low: ...
    high; low: ...
    in all other cases: ...
  ...
```

The principles of live

Live has a **timeline**.
Many **events** happen in live.
Live has a **context**.

These three principles are used throughout the specification of Hippocrates.

The Hippocrates language

DISCLAIMER
All examples below are intended to be illustrative for principles of the language. They are not validated concerning their medical content.

Philosophy of the language

Completeness

When it comes to decision making in the medical space it is important that all options are evaluated. If a script, for example, calculates the dose of a drug based on the bodyweight there should not be any gap or there must be a deliberate decision that all other possibilities are treated the same way.

```
assess body weight:
   0.5 kg ...  5.0 kg: ...
   5.1 kg ... 10.0 kg: ...
  10.1 kg ... 20.0 kg: ...
  in all other cases: ...
```

The definition of body weight would look like this:

```
"body weight" is a value:
```

```
valid units are kg, lbs.
valid values are between (0.5 kg ... 400 kg).
```

Readability

Readability is one of the primary goals of the language. Therefore, no default values are used for any command. In principle each statements should be easy to understand in its context (plus/minus some lines). Here are some examples that help support that goal:

Natural spelling. Currently no programming language exists that let programmers use spaces in either function or variable definitions or even let you you define whole sentences that have meaning. Once Hippocrates learned an expression, it can be used as in natural language (some limitations apply).

Example(s):

"Teaching" Hippocrates that the term "respiratory rate" is a value is as simple as this.

```
"respiratory rate" is a value: ...
```

Once the expression is learned, it can be used like this:

```
assess respiratory rate: ...
```

Values need to come with units. Instead of having a command that says ...

```
...between (10 ... 20)...
```

...Hippocrates will require the author to write:

```
...between (10 kg ... 20 kg)...
```

No implicit default values. In many languages default values are used if a parameter is not given. Hippocrates will always require the full list of parameters in order to ensure that the expression can be understood without deep knowledge of plan that is executed.

Example(s):

While most of the message might address the patient Hippocrates will require the express explicitly who is the receiver of the message.

```
say to patient "Message to Patient".
say to caregiver "Message to Caregiver".
say to physician "Message to Caregiver".
```

>>>>> TO BE REVISED

Integrated conditions: conditions that determine triggering an event can be directly integrated in the statement that is intended to respond to the event. This is easier to read than setting a condition and the catch statement apart from each other.

Example:

The following event handler integrates the condition for trigger (within \period of time\) directly into the call.

```
after contacting \target audience\ [within
  \period of time\] has failed
```

<<<<<

Meaningful

Values always come with a meaning. A meaning that determines healthy from not healthy for example or normal from not normal. And even the same value might have different meaning in a different context. The respiratory rate, for example, has plenty of normal ranges based on the age. Therefore Hippocrates lets you define the meaning of values. This allows physicians to write scripts without the need to know exact boundaries.

Example:

In the example below the respiratory rate (per minute) which is a value is complemented with meanings depending on the age (in years). In case the age is only 0 years the meaning is based on the pediatric age (in months).

```
"respiratory rate" is a value:
  ...
  meaning:
    assess age:
      0 years:
        assess pediatric age:
          0 months ... 2 months:
            assess value:
               0 ...  19: meaning = "critically low".
              20 ...  29: meaning = "low".
              30 ...  60: meaning = "normal".
              61 ...  80: meaning = "high".
              81 ... 100: meaning = "critically high".
  ...
```

Beside the capability to define a meaning also a score can be defined. This can be used for discrete value that can be used for all kinds of assessments (like questionnaires). These score can be used to teach Hippocrates what answers represent a good value and which ones represent a bad value which is useful for statistical analysis.

Example:

The following example scores answers to pain levels, which can be used later on for statistical analysis.

```
"pain level" is a value:
  valid values:
    value is ("No pain"; "Mild pain"; "Moderate pain";
        "Severe pain": value = valid.
...

  scoring:
```

```
    assess value:
      no pain      : score = 0. score is best score.
      mild pain    : score = 10.
      moderate pain: score = 100.
      severe pain  : score = 1000. score is worst score.
  ...
```

Later in a plan this score can be used like that

```
...
diagnosis >
  context >
    timeframe is between (2 weeks ago ... now)!
  assess trend of pain level:
    getting worse: ...
...
```

As an alternative a decision could be based on the majority of answers.

```
...
diagnosis:
  context >
    timeframe is between (2 weeks ago ... now)!
  assess majority of pain level:
    severe pain: ...
...
```

Context

The syntax of Hippocrates is designed in a way that a statement always creates the context for subsequent statements.

Example:

A message creates the context for parameters that control the message. **show message** creates the context for the following two lines.

```
show message to patient "This is a message for a patient":
  message expires after 30 minutes.
  color = red.
```

Where this approach is not possible as the context creation contains more than one statement it can be created explicitly.

Example:

There are more complex cases where the context can be created explicitly for a subsequent block. The following shows how a context for an analysis can be created. In this case

```
diagnosis:
  context:
    timeframe is between (2 weeks ago ... now)!
```

```
    drug taken is yes!
  assess majority of pain level:
    severe pain:
      assess count of drug taken:
        0:
...
```

Separation

Separation is another primary design goal of Hippocrates. This separation comprises two domains.

Separation of technical domain. The language contains nothing that is specific to any platform the language is running on. Therefore no knowledge about technology is necessary.

Automatic handling of data. All scripts that are created are created for a single person. Although the Hippocrates engine can deal with multiple plans and person the author of a script never has to cope with that complexity.

Constraints

Constraints are another concept on Hippocrates. This means that every value must have a definition of valid values, which could also be a range or a set of ranges as well as discrete values. This eliminates to danger of assigning values that would lead to faulty and potential dangerous decisions.

Values need to have a range. This range is used ensure that only meaningful and valid values are used at any time in the script.

```
"body weight" is a number:
  valid units:
    valid units are (g; kg; lb).
  valid values:
    assess value:
      0.5 kg ... 200.0 kg: value = valid.
  ...
```

History

Keeping information about the history is an important principle of Hippocrates. Basically this has two meanings. Values that have been created in the past cannot be changed and values that are calculated will be stored with reference to values that have been used for calculation.

Local

The Hippocrates language is designed to automatically translate scripts into supported local languages. Therefore each Hippocrates expression comes with pre-defined translations. Furthermore all text values can be complemented with translations.

Medical

Hippocrates is focused on the medical domain and comes with a basic knowledge of medical terms. These terms can be used to associate values with anatomy or medicine. These terms will help implementing appropriate user interfaces.

Persistent

Hippocrates automatically stores all changes to values. There is no need for the author of a script to take care of storing data.

Autonomy

A plan is always complete. This means that any plan comes with all definitions that are required to execute the plan. Once the plan is deployed to Hippocrates engine it is analyzed and duplication is removed to save storage.

Common language definition

Conventions

The following conventions are used in this document to specify the Hippocrates language.

```
{definition of grammar}::
      The definition

\instance of grammar\

{{\repeated section\}}

[\optional element\]
```

Discussion:
Is the following really required? The same principle as templates.

If an element in a definition is further specified the syntax looks like this:

```
{definition of grammar <argument>[{{;
      <argument>}}]}::

      {argument}::
            {<expression to be replaced> = <replacing expression>}
```

Basic syntax

The basic syntax for Hippocrates is designed to follow the sentence structure of natural languages.. Furthermore expression can be written in a way that they also read like natural language. Hippocrates supports international character sets (UTF-16).

```
{dot}::
      .

{colon}::
      :

{question mark}::
      ?

{exclamation mark}::
      !

{line end}::
      \dot\|\colon\|\question mark\|\exclamation mark\

{slash}::
      /

{comma}::
      ,

{blank}::
      U+0020

{new line}::
      U+000A

{hyphen}::
      -

{quotation mark}::
      "

{semicolon}::
      ;

{opening round bracket}::
      (

{closing round bracket}::
      )

{digit}::
      0, 1, 2, 3, 4, 5, 6, 7, 8, 9

{character}::
      \Upper and lower-case characters in supported languages\|\digit\

{word}::
      {{\character\}}

{word seperator}::
      \blank\|\comma\

{words}::
```

```
      \word\\word seperator\\word\[{{\word seperator\\word\}}]

{paragraph name}::
      \quotation mark\\words\\quotation mark\\blank\\words\\line end\

{instruction}::
      \words\\dot\

{question end}::
      \question mark\

{question}::
      \words\\question end\

{constraint end}::
      \exclamation mark\

{constraint}::
      \words\\constraint end\

{indent}::
      \blank\\blank\

{continuation indent}::
      \blank\\blank\\blank\\blank\

{paragraph}::
      [\paragraph name\]{{\word\}}
        [\blank\{{\colon\{{\word\\blank\}}}}]\colon\\new line\
        \indent\]{{\instruction\|\paragraph\}}

{enumeration}::
      \words\\semicolon\\words\[{{\semicolon\\words\}}]

{ellipsis}::
      ...

{range}::
      \words\\ellipsis\\words\

{expression}::
      \words\

{formula}::
      \expression\ |
      [\opening round bracket\]\expression\\mathematical
            operator\\expression\[\closing round bracket\] |
      [\opening round bracket\]\formula\\mathematical operator\\formula\
            [\closing round bracket\]
```

Rules:

1. When a \formula\ is evaluated mathematical rules apply.

Common definitions

{integer number}::

```
    0, 1, 2, 3, 4, 5 ... 1000000000
    null, one, two, three, four, five, six, seven, eight, nine, ten,
          eleven, twelve
```

{positive integer number}::

```
    1, 2, 3, 4, 5 ... 1000000000
    one, two, three, four, five, six, seven, eight, nine, ten,
          eleven, twelve
```

{positive integer number greater than 1}::

```
    2, 3, 4, 5 ... 1000000000
    two, three, four, five, six, seven, eight, nine, ten, eleven, twelve
```

{enumerator}::

```
    1st, 2nd, 3rd, 4th, 5th, 6th, 7th...
    first, second, third, fourth, fifth, sixth, seventh, eighth, ninth,
          tenth, eleventh, twelfth
```

{special value}::

```
    unknown
    none
    infinite
```

>>> FOR DISCUSSION

valid

invalid

Is definition of valid and invalid necessary or will the other word be sufficient

<<<

Rules:

1. *unknown* is a special value that can be used for any type of value in case the value is unknown or cannot be calculated. This is for example used is a question is aborted.
2. *none* is a special value that is used if a question leads to this answer (e.g. if the *number of future occurrences of <period>* is requested, but there is none).
3. *infinite* is returned if no limit can be evaluated for a value, but the value exists (like the the number of breakfast in the future).

{frequency}::

```
    once
    twice
    \positive integer number greater than 1\ times
```

{percentage}::

```
     0%, 1%, 2%, 3% ...
```

{percentage100}::

```
    0%, 1%, 2% ... 100%
```

{minute}::

```
     0, 1, 2, 3, 4, 5 ... 59
```

{hour24}::

```
    0, 1, 2, 3 ... 23
```

{hour12}::
1, 2, 3 ... 12

{time of day}::
\hour24\:\minute\
\hour12\:\minute\ am
\hour12\:\minute\ pm
\hour12\ am
\hour12\ pm

{date}::
\day of month\\date delimiter\\month\\date delimiter\\year\
\month\\date delimiter\\day of month\\date delimiter\\year\

{day of month}::
01, 02, 03, 04 ... 31 (with automatic adjustment to actual length of <month> in <year}>

{month}::
January, February, March, April, May, June, July, August, September, October, November, December.

{year}::
\min year\, 1901, 1902 ... \max year\

{min year}::
1900

{max year}::
2200

{date delimiter}::
:
-
/

Discussion:
1. Will the parser be able to safely distinguish between <date delimiter> and <mathematical operator>?

{day of week}::
Monday, Tuesday, Wednesday. Thursday, Friday, Saturday, Sunday, first day of week, last day of week.

{time unit}::
year | years
month | months
week | weeks
day | days
hour | hours
minute | minutes
second | seconds

Rules:
1. Plural of {*time unit}* is used for numbers other than 1 (one).

```
{point in time}::
      \month\
      \month\ day \day\
      \month\ day \day\ at \time of day\
      \day of week\
      \day of week\ at \time of day\
      \time of day\
      \date\
      \date\ at \time of day\
      now
      [\period of time\ \before|after\] \begin|end\ of \period\
      [\period of time\ \before|after\] \event\
      \period of time\ ago
```

Discussion(s):

1. It needs to be carefully evaluated, if <point in time> related to <period> or <event> can always be used as these might not be uniquely identifiable.

   Potential solutions:

   1. PREFERRED: Restrict use of referral to <period> or <event> to uniquely identifiable points in time.
   2. Do not allow referral to <period> and <event>

```
{period of time}::
     1 minute | \number of minutes\ minutes
     1 hour | \number of hours\ hours
     1 day | \number of days\ days
     1 week | \number of weeks\ weeks
     1 month | \number of months\ months
     1 years | \number of years\ years
```

Rules:

1. Plural is used for *<number of ...>* other than 1 (one).

The range each period can take is in a range that switching to the next higher dimension will cause about a 1% error. An exception is made for years which will be limited to 100 years.

```
{number of minutes}::
     1, 2, 3, 4 ... 6000

{number of hours}::
     1, 2, 3, 4 ... 2400

{number of days}::
     1, 2, 3, 4 ... 700

{number of weeks}::
     1, 2, 3, 4 ... 5200

{number of month}::
     1, 2, 3, 4 ... 1200

{number of years}:::
     1, 2, 3, 4 ... 100
```

```
{timeline direction}::
    past
    future

{mathematical base operator}::
    +
    -
    *
    /
    ^
    Square root? could be represented as ^0.5?? (Readable?)

{value reference}::
    initial
    current
```

Periods and events have some values that can either refer to the *initial* definition or the *current* value.

```
{predefined unit}::
     \temperature unit\
     \length unit\
     \weight unit\
     \volume unit\
     \time indication unit\
     \time reference unit\

     {temperature unit}::
          °F
          °C

     {length unit}::
          km
          m
          mm
          mile
          inch
          foot

     {weight unit}::
          mg
          g
          kg
          lb

     {volume unit}::
          oz
          l
          ml
          fl oz
          gal

     {time indication unit}::
          week
```

```
      day
      hour
      min
      sec
```

{time reference unit}::

```
      per week
      per day
      per hour
      per min
      per sec
```

Rules:
1. Units *°F, °C, km, m, mm, mile, inch, foot, mg, g, kg, lb, oz, l, ml, fl oz, gal, week, day, hour, min, sec, per week, per day, per hour, per min* and *per sec* can be used with numbers only.
2. Units *[time of day]* and *[date]* can be used with time indications only.
--> SPELL OUT RULES
3. For °F, °C predefined conversion rules are defined.
4. For km, m, mm, mile, inch, foot predefined conversion rules are defined
5. For mg, g, kg, lb, oz predefined conversion rules are defined.
6. For l, ml, fl oz, gal predefined conversion rules are defined.
7. For week, day, hour, min, sec predefined conversion rules are defined.
8. For per week, per day, per hour, per min, per sec predefined conversion rules are defined.
<--

Comment(s):
1. Concentrations like mmol/L or mg/dL are not defined as they require specific conversion rules that need to be defined each time these units are used in an interchangeable way.

Teaching paragraph names

Hippocrates can learn new expressions using *<text>* that are used later on as *<string>* expressions.

Rules:
1. Any vocabulary must be unique.
2. A vocabulary cannot be contained in another vocabulary (example: "infection" and "serious infection". In this case the first definition is contained in the second definition).

Discussion:
1. In theory even this problem could be addressed if Hippocrates starts identifying vocabulary from the longest expression down to the shortest expression.

Example(s):

```
"diastolic blood pressure" is a number...
```

In the script the new expression can be used without the parenthesis.

```
assess diastolic blood pressure (
  ...
```

Defining values

More description of values.

Hippocrates plans can contain any number of values.

```
{name of value}::
      \text\

{type of value}::
      a number
      an enumeration
      a time indication

{value}::
      \name of value\ is \type of value\ (
        {{\paragraph\}}
      ).
```

Assigning values

Hippocrates uses the equal sign ("=") o assign values.

```
{Lvalue}::
      \value\

{Rvalue}::
      \a number\
      \an enumeration\
      \a time indication\
      \value\
      \formula\

{value assignment}::
      \Lvalue\ = \Rvalue\.
```

Rules:
1. An assignment always creates a new instance of **\Lvalue**.
2. **\LValue** and **\Rvalue** must have the same unit or units that can be converted into each other.
3. **\Lvalue** must be **\a number**, **\an enumeration** or **\a time indication**.
4. **\Rvalue** can only assigned to **\Lvalue** if no calculation rule is defined in **\Lvalue**.

Use of units

Hippocrates can handle units associated with numbers and time indications. Unit handling can be used in value definitions and questions.

Definition of units

```
{unit definition}::
      assess unit >
        {{\unit assessment paragraph\}}<

      {unit assessment paragraph}::
            \unit\[{{; \unit\}}] (
              \unit is valid \
              {{\unit conversion rule\}}
            ).

      {unit}::
            \text\
            \predefined unit\

      {unit is valid sentence}::
            unit is valid.

      {unit conversion rule}::
            1 \unit (1)\ = \formula\\unit (2)\.
            1 \unit (2)\ = \formula\\unit (1)\.
```

Rules:
1. Each **\unit** can be used only once.
2. For each unit that should be converted into another unit a **\conversion rule** in both directions within the same **\unit assessment paragraph** must be defined.
3. If no **\conversion rule** is defined all values that refer to each other must use the same unit.
4. **\unit handling** is possible for **\a number** and **\a time indications**.
5. If one or more units are specified for a **\value** one of these specified units must be used in the script next to the value.
6. If the intended use of a \value\ is a concentration there must be a conversion rule between mmol/L and mg/dL defined.

Example(s):

The following example defined a dose that can have different units. When a specific unit is selected all other values that refer to that value must use the same unit.

```
"dose" is a number >
  valid units >
    assess unit >
      "tablet"; "mg"; "drop" > unit is valid. <<
  ...
<

...
dose = 1 tablet.
...
```

However, units can also be converted into each other by providing formulas to achieve that.

```
"glucose level" is a number >
  valid units >
    assess unit >
      "mmol/L"; "mg/dL" >
        unit is valid.
        1 mmol/L = 18.02 mg/dL.
        1 mg/dL = (1/18.02) mmol/dL. <<<
  ...
<
```

Accessing the unit definition

{unit accessor}::

```
unit of \value\ -->
      \specified unit\[{{; \specified unit\}}]
```

Rules:
1. **\unit of \value** can only be used if a unit is defined.

Examples:

It could be the case that a value can have different units that cannot be converted into each other and therefore different actions need to be taken based on the unit used.

```
assess unit of daily dose >
  tablets > {{\paragraph\}} <
  drops > {{\paragraph\}} <<
```

{unit converter accessor}::

```
(\value\ in \specified unit\) -->
      \value\ \specified unit\
```

Rules:
1. **\\value\ in \specified unit** can only be used if a unit is defined and conversion rules are defined

Example(s):

A BMI needs to be calculated in metric units.

```
BMI = (weight in kg)/(height in m)^2.
```

Selecting ranges of values

Range selections can be used to increase readability is larger groups of numbers needs to used for decisions (e.g. performing a selection based on weight).

{range selector}::

```
\range\[{{;\range\}}]
```

```
{range}::
      \valid value\
      \meaning of value\
      \score of value\
      \lower range\ ... \upper range\

      {lower range}::
      {upper range}::
            \valid value\
            \score of value\
```

Rules:
1. **\value** must be **\a number** or **\a time indication**.
2. \lower range\ must be less than \upper range\. If the associated value is an enumeration scores must be defined.
3. \meaning of value\ is only possible if best value and worst value are defined.

Example(s):

The following example is based on a simple range selector also demonstrating how ranges can be separated from each other (as they cannot overlap). The range values used must be within the valid range of *body weight*.

```
assess body weight >
  0.5 kg ... 9.9 kg > ... <
  10.0 kg ... 19.9 kg > ... <
  ...
<
```

Making assessments and diagnosis

Decision will be based on assessment of values. Hippocrates provides an efficient mean to assess either single values or value groups or multiple values or value groups with as little text as possible.

```
{assessment and diagnosis}::
      \assessment\ |
      \diagnosis\

      {assessment}::
            assess \value\ >
              {{\range selector\ > {{\paragraph\}} <}}
              [<other cases>.]
            <

      {diagnosis}::
            diagnosis >
              criteria >
                \assessment question\?[{{
                \assessment question\?}}]
              assess number of applying criteria >
                {{\range selector\ > {{\paragraph\}} <}}
```

```
        [for all other values of \value in question\{{,
          \value in question\}} >
            \catch all other cases\<]
      <
```

Discussion:
1. Exceptions for *<other cases>* need to be defined. However, this should only not be required if the assessor is used for selection or building definitions.
2. Alternative/additional syntax instead of using **assess \value**:

```
      in case \value\ is \value criteria\ > ... <
      for all other values of \value\[{{; \value\}}] > ... <
```

Rules:
1. **\for all other values of** is required if the criteria do not cover all possible values that a value in a question could have. This ensures that all other possibilities are actively taken into account.

```
{assessment question}::
      \value\ is \value criteria\

{value criteria}::
      \valid value\
      \meaning of value\
      \criteria selector\ (\criteria value\[{{; \criteria value\}}])
      between (\lower range\ ... \upper range\)

      {criteria selector}::
            \positive integer number\ of
            at least \positive integer number\ of
            not more than \positive integer number\ of
            all of
            none of
            between (\lower range\ ... \upper range\) of

      {criteria value}::
            \valid value\
            \meaning of value\

{other cases}::
      in all other cases >
        \catch all other cases\ <

      {catch all other cases}::
            no action |
            {{\paragraph\}}

{number of applying criteria}::
      number of applying criteria >> (complete decision)
            \integer number\
```

Contains the number of *{assessment question}* that applied in the assessment.

Rules:

1. If assessment are nested **\number of applying criteria** contains the value for the related paragraph.
2. Valid range starts with 0 and ends with number of \assessment question\s used.

{applying criteria}::

```
applying criteria >>
      \name of value\[{{; \name of value\}}]
```

Returns the assessment if the criteria for **\value** applied without having to repeat the conditions. The respective value can be addressed by using a *{value assessor}*.

Rules:
1. If **\value** is used more than one **\assessment question** and only one of the **\assessment question** applied for **\value** the **criteria for \value** is true.

Example(s):

The following examples show the use of a **\simple assessor** and a **\question based assessor**.

```
assess systolic blood pressure >
  normal                      > ... <
  low; high                   > ... <
  critical high; critical low > ... <<

diagnosis >
  criteria >
    pain in left arm is one of (none; moderate)?
    pain in right arm is one of (none; moderate)? <
  assess number of applying criteria >
    0 ... 1 > no action. <
    2       > show message to patient
                "Please come an see us.". <<
  for all other values of pain in left arm,
    pain in right arm > no action. <
<
```

The following example has a total of 4 questions in the decision criteria. If between 2 and 3 of them have the meaning *mild* or *moderate* the overall assessment of the criteria will be true. In the following block the actual number of applying criteria can be accessed using *number of applying criteria*.

```
diagnosis >
  criteria >
    left arm pain is one of (mild; moderate)?
    right arm pain is one of (mild; moderate)?
    left leg pain is one of (mild; moderate)?
    right leg pain is one of (mild; moderate)? <
  assess number of applying criteria >
    1       > show message to patient "Don´t worry". <
    2 ... 3 > show message to patient "Could be better". <
    4       > show message to patient "You should come
                and see us". <<
```

```
        for all other values of left arm pain, right arm pain,
          left leg pain, right leg pain > no action. <
      <
```

The example from above could there be written like this to further assess if the pain is in both arms.

```
      ...
      assess number of applying criteria >
        2 >
          assess applying criteria >
            left arm pain; right arm pain > ... <
            in all other cases            > ... <<
        in all other cases > ... <
      <
      ...
```

Dealing with uncertainty

Values that are used in decision might not always be 100% certain. This can have different reasons from aging of value or values that are close to the boundaries of these values. Hippocrates complements every value with a confidence value between 0% and 100%. This can be used to make sure that decisions are based on values that are trustworthy.

```
{confidence controlled paragraph}::
      \confidence selector\ >
        {{\confidence range selector\ >
        {{\paragraph\ >
          \innermost paragraph\ <
        }}<}}
        [\other cases\]
      ).

      {confidence selector}::
            confidence of values >> (complete decision)
                  0% ... 100%

      {confidence range selector}::
            \lower confidence level\ ... \upper confidence level\

            {lower confidence level}::
            {upper confidence level}::
                  \percentage100\
```

Rules:

1. If **\confidence controlled paragraph** is used Hippocrates checks each value that is used if it is within the specified confidence range. Once the final decision point, the **\innermost paragraph** is reached all value that have been used in the decision will be checked again. Only if all of these values are still in that range the **\innermost paragraph** is executed.
2. **\lower confidence level** must be less than **\upper confidence level**.
3. If not all possible confidence levels are addressed by using **\range selector** a **\other cases** paragraph is necessary.

4. **\confidence controlled paragraph** cannot be nested.
5. Whenever a **\confidence selector** is found Hippocrates will do the following:
    a. Identify values that are used in the current level (no subsequent paragraphs will be evaluated).
    b. Check all values on their confidence level and for those which are below the highest threshold issue a question.
    c. Once the **\innermost paragraph** is reached re-assess the confidence level of all values that have been used on the path to that **\innermost paragraph**. If this fails the next lower **\confidence range selector** will be chosen.
    d. Any **\value** that is changed in the **\innermost paragraph** will have the lowest confidence used on the path to the **\innermost paragraph**.

Example(s):

Making a decision that changes the dose based on *weight* and *age* requires a decent level of confidence in that value.

```
...
assess confidence of values >
  80% ... 100% >
    assess age >
      0 years ... 8 year >
        show message to caregiver "Do not use for children".
        single dose = 0 tablets. <
      9 years ... 150 years >
        assess weight >
          0.5 kg ... 15 kg >
            show message to patient "For persons under 15 kg this
              drug should not be used.".
            single dose = 0 tablets. <
          16 kg ... 25 kg >
            single dose = 2 tablets. <
          26 kg ... 250 kg >
            single dose = 3 tablets. <<<
     <
  0% ... 79% >
    single dose = 0 tablets.
    show message to patient "The confidence of the data is not
      sufficient to calculate a dose. Please ask your physician
      for your dose." <$
<
...
```

Alternative syntax:

```
in case confidence of values is between (80% ... 100%) >
  assess age > ... <<
for all other values of confidence of values > ... <
```

Defining periods of time

{period timeline}::

```
between(\period start\ ... \period end\
      [{{; \period start\ ... \period end\}}])

      {period start}::
      {period end}::
            \point in time\
```

Rules:
1. If **\period start** is a **\date** **\period end** must be after **\period start**.
2. If **\period start** is prior **\period end** and a flip-over will be calculated.

Example:

Simple period timeline definitions.

```
      between (Monday ... Friday;
        09:00 am ... 12:00 pm; 02:00 pm ... 06:00 pm)

      between (January ... April)

      between (10-Apr-2014 ... 20-Apr-2014)
```

Example:

Period timeline definition that cause a flip-over.

```
      between (Wednesday ... Monday) >>
            Wednesday, Thursday, Friday, Saturday, Sunday, Monday

      between (April ... January) >>
            April, May, June, July, August, September, October,
            November, December, January
```

Period timeline definition helpers

In some cases it might be more natural to use other expressions to define a period.

```
{for period of time}::
      for \period of time\ >>
            between (\begin of paragraph\ ...
              \begin of paragraph\ + \period of time\)


{until point in time}::
      until \point in time\ >>
            between (\begin of paragraph\ ... \point in time\)

{until event}::
      until \event\
```

Rules:

1. If no end point can be calculated Hippocrates will use the end of **\max year**.

Examples:

The following examples use different reference points to calculate the period start. The first example starts running once the plan is activated. The second example starts running on the day of a surgery

```
plan >
  for 9 days >
    ...

plan >
  "surgery aftercare" on 24-Sep-2015 for 1 year >
    ...
```

Also end points can be defined that be defined using **\point in time** or an **\event**.

```
until 20-Dec-2014
```

Another period could also be used as a referral point

```
until end of rehabilitation
```

An event could also be used. In this case a custom event would signal that a certain training level is reached.

```
until rehabilitation level 2 has been achieved
```

Defining context

A context can be used to narrow down possibilities in a subsequent paragraph. This can either be on the timeline, a specific value group or a drug.

At a high level a context is a construct that allows narrowing down of values for analytical purposes and to restrict access to data.

```
{context}::
      \data context\
      \diagnosis context\
```

Rules:
1. If a *<question>* is used to create a context *for analysis* must be specified,
1. If any analytical method should be used *<context purpose>* must be specified.

```
{data context}::
      context is \data reference\ >
        {{\paragraph\}}
      ).

      {data reference}::
            \value\ |
```

```
(\value\; \value\[{{; \value\}}]) |
\value group\ |
(\value group\; \value group\[{{; \value group\}}]) |
all value groups |
\drug\
```

Rules:
1. **all value groups** can be used to consolidate a value that is used in multiple value groups. Any value that is assessed under that context must not have the intended use of **for use in value groups only**.

{diagnosis context}::

```
diagnosis >
  context >
    \context item\[{{\context item\}}] <
  {{\paragraph\}}
<
```

Rules:
1. A **\filter context** must always be the first statement in a new **\paragraph**.
2. **\context item** cannot overlap.
3. A context applies for all subsequent paragraphs in the paragraph it is defined.
4. **\timeframe is \time scope** only works in the past until **now**. If the end point (e.g. defined by a date) is still in the future the context cannot be created and therefore **{{\paragraph\}}** does not get executed.
5. The actually calculated time scope is based on the shortest period that fulfills all **\time scope** criteria. This could potentially end in no data in that period and would leave values that are calculated based on that timeframe as unknown.
6. Contexts with **\timeframe is \time scope** can be nested as long as the timeframe in the inner context is within the timeframe of the outer context.
7. If a \filter context item\ is not used in the **\paragraph** or any subsequent **\paragraph** it cannot be defined.

{context item}::

```
\value filter\! |
data is \data reference\! |
timeframe is \time scope\! |
drug is \drug\!
```

The next section requires full description.

{data context}::

```
data is \data reference\! |
data is \data reference\! >
  timeframe is \time scope\. |
  use first|last \positive integer number\
    data points.
```

{value filter}::

```
\assessment question\
```

Rules:
1. **\value filter** only has impact on analytical methods.

{time scope}::

```
\period timeline\
during \period\
outside \period\
```

Discussion:

1. There must be means to cover the fact that the context cannot be created.
   Potential solutions:
   a. At any given point in the diagnosis paragraph there must be an event handler that could catch that fact.

```
in case context creation failed >
  ... <
```

To be tested:

The following examples should count the number of data points with a 2 weeks period. What is tested, if for both values 10 data points exist.

```
diagnosis >
  context >
    timeframe is between (2 weeks ago ... now)! <
  in case context creation failed >
    do something. <
  assess >
    count of age is 10?
    count of usage is 10? <
  in case number of applying criteria is 2 > ... <
  for all other values of applying criteria > ... <
<
```

Example:

The following example shows the assessment of blood pressure that consists of two values that are grouped.

```
"systolic blood pressure" is a number >
  ...
<

"diastolic blood pressure" is a number >
  ...
<

"blood pressure" is a value group >
  grouped values are (systolic blood pressure;
    diastolic blood pressure).
<

plan >
  context is blood pressure >
    assess diastolic blood pressure >
      hypotension >
        assess systolic blood pressure >
          hypotension        > ... <
          normal             > ... <
          prehypertension    > ... <
          in all other cases > ... <<
```

```
        normal > ... <
        prehypertension > ... <
        in all other cases > ... <<<
    <
```

Example:

A very simple time scope that starts with the plan and end now.

```
diagnosis >
  context >
    timeframe is between (begin of plan ... now)! <
  assess count of ...
```

The following expression would limit all subsequent data access operations to values that have been created between now and 2 weeks ago during breakfast. And this could be a clever trick, if breakfast is not a fixed period but the begin and end of that period could be edited. Therefore it would be possible that 14 different timeframe are actually used for filtering.

```
diagnosis >
  context >
    timeframe is between (2 weeks ago ... now)
      during breakfast! <
  assess count of ...
```

This example could also be written as if the time during and outside the breakfast period needs to create different contexts:

```
diagnosis >
  context >
    timeframe between (2 weeks ago ... now)! <
  diagnosis >
    context >
      timeframe is during breakfast! <
    assess count of ...
  <
  diagnosis >
    context >
      timeframe is outside breakfast! <
    assess count of ...
  <
<
```

Example:

In the following example the values of a continuous blood pressure management will be summarized in an average value.

```
after blood pressure measurement >
  context >
    data is blood pressure!
    timeframe is between (5 minutes ago ... now)! <
```

```
    assess average of systolic blood pressure >
      20 mm/Hg ... 60 mm/Hg > ... <
        ... <
    <
  <
```

To be a bit more in the safe side one could also check the number of data points that have been collected in these past 5 minutes. Let's assume that we have requested a reading every 10 seconds, we should see 5 x 6 = 30 readings. If 10% are lost, it would still be sufficient.

```
  after blood pressure measurement >
    diagnosis >
      context >
        data is blood pressure!
        timeframe is between (5 minutes ago ... now)! <
      assess count of systolic blood pressure >
        27 ... 30 >
          assess average of systolic blood pressure >
            20 mm/Hg ... 60 mm/Hg > ... <
            ... <<
        in all other cases >
          show message to patient "The blood pressure reading
            did not collect enough data points. Only "
            count of systolic blood pressure " data points have
            been collected. We need at least 27 in this 5 minute
            period. Please repeat the reading."

          ask "Please sit down and confirm and confirm
            once the device ready" >
             assess answer >
              "Device is ready" >
                blood pressure device is ready. <
              "I don't want to measure now" >
                no action. <<<<<<
  <
```

Example:

In the following example the performance for certain daily activities is assessed and a verbal assessment is stored in a value. By using *assess timeline* a filter is created that is used for the calculation of *total of duration of exercise*. (see "Statistical expressions"). The example combines time scope and value group scope.

```
  "duration of activity" is a number >
    valid units >
      assess unit >
        min, minute, hr, hour > unit is valid. <<<
    valid values >
      assess value >
        0 min ... 24 hr > value is valid. <<<
  <

  "indoor activity" is an enumeration >
```

```
    intended use >
      for use in value groups only. <
    valid values >
      assess value >
        "swimming"; "tennis" > value is valid. <<<
  <

  "outdoor activity" is an enumeration >
    intended use >
      for use in value groups only. <
    valid values >
      assess value >
        "skiing"; "football" > value is valid. <<<
  <

  "duration of indoor activity" is a value group >
    grouped values are (indoor activity; duration of activity).
  <

  "duration of outdoor activity" is a value group >
    grouped values (outdoor activity; duration of activity).
  <
```

The duration of all activities in both groups should be calculated as follows.

```
  diagnosis >
    context >
      timeframe is between (1 week ago ... now)!
      data is (duration of outdoor activity;
        duration of indoor activity) <
    show message to patient "Your total activity
      in past 7 days was"
      total of duration of activity in minutes.
  <
```

If only outdoor activities should be counted the expression would be:

```
  diagnosis >
    context >
      timeframe is between (1 week ago ... now)!
      data is duration of outdoor activity! <
    show message to patient "Your total outdoor activity
      in past 7 days was "
      total of duration of activity in minutes.
  <
```

Now lets go even one level deeps and let's see how much time the person spent playing football. Do not forget to mention that

```
  diagnosis >
    context >
      timeframe is between (1 week ago ... now)!
      data is duration of outdoor activity!
      outdoor activity is football! <
    assess total of duration of activity >
```

```
        0 min ...   100 min >
          2 weeks football performance = poor. <
      101 min ...   600 min >
          2 weeks football performance = nice. <
      601 min ...  2000 min >
          2 weeks football performance = professional. <
     2001 min ...  4001 min >
          2 weeks football performance = champions league. <
     in all other cases >
          2 weeks football performance = unrealistic. <<
<
```

Handling violations of constraints

Discussion:
1. This paragraph needs to be reconsidered as exception handling might be build into event handling.

Each constraint that can be defined will also have to possibility to handle violations. This could be setting a duration or period under the minimum duration or the other way round.

```
{constraint violation handling}::
     in case of constraint violation >
       {{\paragraph\}}
     <
```

Rules:
1. **{constraint violation handling}** is only available within constraints.

Example(s):

A treatment period is specified for a certain period of time and can be extended in the script based on feedback. However, to avoid an extension that goes beyond a certain period of time the treatment should stop.

```
"treatment" for 9 days >
  maximum duration of treatment is 15 days >
    in case of constraint violation >
      show message to patient "Your treatment has been extended
        to a maximum duration of 15 days. Based on your feedback
        the symptoms seem still be present. Please make an
        appointment with us with next 2 days."
      stop plan. <<
<

Alternative version

"treatment" for 9 days >
  in case duration of treatment gets shorter than 15 days >
    show message to patient "Your treatment has been extended
      to a maximum duration of 15 days. Based on your feedback
      the symptoms seem still be present. Please make an
      appointment with us with next 2 days."
```

```
        stop plan. <
    <
```

~~Age of values~~

~~**As Hippocrates manages age of values through the mean of reuse period it needs to be discussed if this expression could lead to complication of scripts.**~~

~~Hippocrates add a timestamp to each value therefore the age of each value (time of last update) can be determined. This functionality is used internally if a reuse period for a value is defined.~~

~~The sub-sentence accessing the age of a value is:~~

```
    {part of sentence to access age}::
        ...age of {value} is {period of time}...
```

~~Rules:~~
~~1. If *{value}* is not valid unknown will be returned.~~
~~2. Use of *age of {value}* does not trigger a calculation of *{value}*.~~

~~Examples:~~

```
    if age of BMI is less than 1 month then (
      ...
    ).
    in all other cases of age of BMI (
      ...
    ).
<<<
```

Statistical analysis

Statistical methods can only be used in a diagnostic paragraph.

```
diagnosis >
  \statistical method\

{statistical analysis}::
    count of
    total of
    min of
    max of
    average of
    median of
    trend of (going up/down; stable; getting better/worse)
    change of
    majority
    minority
```

```
within bottom X
within top X
correlation
```

Slice values into chucks and then create values for average, median

```
average of 5 minutes average of <value>
median of 5 minutes median of <value>

average of daily average of <value>
average of weekly average of <value>
average of monthly average of <value>
average of yearly average of <value>

trend of this
trend of that
```

Interacting with the user

Defining addressees

An addressee can be any person (or even system) that should receive information or is requested to provide information.

```
{addressee}::
      \text\ is an addressee >
        {{\addressee paragraph\ >
          {{\paragraph\}} < }}
      <

      {addressee paragraph}::
            [\enumerator\ contact information >] |
            [contact information is missing >] |
            [before consent is requested from patient >] |
            [after consent has been rejected by patient >] |


            \enumerator\ contact information >
              \contact information\ [>
                [after contacting has failed >
                  {{\paragraph\}} <]

                [after contacting has succeeded >
                  {{\paragraph\}} <]<]
            <

                  {contact information}::
                        \Hippocrates contact\ |
                        \mail contact\

                        {Hippocrates contact}::
                              Hippocrates ID is \text\
```

```
{mail contact}::
      mail address is \text\


after contacting has failed >
  {{\paragraph\}} <
```

In this paragraph one specific action and one specific value are available.

```
retry with \enumerator\ contact information [
  after \period of time\].
```

With that statement any contact information can be referred to as the next to be used.

```
number of failures >> (complete decision)
      1 ... X
```

Contains the number of failed attempts for **\enumerator\ contact information**.

Rules:

1. If **\enumerator\ contact information** is missing **contact information is missing** must be defined. In case no contact information is provided in the script the user must enter this information in the application Hippocrates is embedded in.
2. Each contact information in **\enumerator\ contact information** must be unique, meaning an addressee can have multiple mail addresses but each of them must be unique.
3. **\Hippocrates contact** can be used only once for each **\addressee**.
4. If **after consent has been rejected by patient** is called no information will be sent to **\addressee**. It should be evaluated if the script should be stopped. All information that should be sent to that **\addressee** will be stored in the log. The script will stop automatically if the **\addressee** is supposed to answer questions.
5. If **after contact failure** is not used Hippocrates will use the contact information that are provided in the order given without any delays between the attempts.
6. **patient** is a reserved expression for an addressee name.
7. In **after contact failed** or **after consent has been rejected by patient** no action can address the same **\addressee** that has failed to be contracted or the consent was not given.

Example(s):

In case a physician would like to get regular feedback on how the patient is doing, he can be defined as addressee.

```
"physician“ is an addressee >
  first contact information >
    mail address is "mail@anydomain.com". >
      after contacting has failed >
        show message to patient "Your physician cannot be
         contacted via mail.". <<<
  after consent has been rejected by patient >
    show message to patient "In case no consent to send information
      to your physician is given, no or limited remote support
```

```
            will be available.". <
      <
```

Defining addressee groups

For many care scenarios it is of importance to define a group of people that is given a task. This can either be a task that they do together or it could be a sequence of addressee that get contacted if the e.g. the first addressee cannot be reached.

```
{addressee group}::
      \addressee group\ is an addressee group >
        grouped addressees are >
          \addressee\; \addressee\[{{; \addressee\}}]. <
        order of contacting >
          {{[\decision\ >]
            \contact logic\ <[<]}}
          [{{\contacting\}}]
          [{{\acknowledging\}}]
      <

      {contact logic}::
            \parallel contact\ |
            \serial contact\

            {parallel contact}::
                  contact all addressees in parallel >
                    [{{\contacting\}}]
                    [{{\acknowledging\}}]
                  <

            {serial contact}::
                  sequence of contacting is (\addressee\; \addressee\[{{;
                    \addressee\}}] >
                    [{{\contact\}}]
                    [{{\confirm\}}]
                  <


                  {contact}::
                        after contacting \target audience\ [within
                          \period of time\]has failed
                            > ... <
                        in all [other] cases of contacting failure
                          > ... <

                        after contacting \target audience\ has succeeded
                          > ... <
                        in all [other] cases of contact success
                          > ... <

                  {confirm}:: (version 2)
                        after confirming contact with \target audience\
                          [within \period of time\] has failed
                            > ... <
```

```
in all [other] cases of confirmation failure
  > ... <

after confirming contact with \target audience\
  has succeeded
    > ... <
in all [other] cases of confirmation success
  > ... <


{target audience}::
      \addressee\
      (\addressee\; \addressee\[{{; \addressee\}}])
      an addressee
      all addressees

stop contacting

contacted addressee

next addressee
```

Examples:

The following examples show how to setup a protocol of a contact group that first tries to reach a professional care persons and if none of them responds tries to reach out to a child of the patient.

```
"care team" is an addressee group >
  grouped addresses are >
    nurse 1; nurse 2; family caregiver. <

  order of contacting >
    assess time >
      08:00 am ... 06:00 pm >
        sequence of contacting is (nurse 1; nurse 2;
          family caregiver). <
      06:01 pm ... 07:59 am >
        sequence of contacting is (nurse 2; nurse 1;
          family caregiver). <<
    after confirming contact with nurse 1 within 10 minutes
      has failed >
        show message to patient nurse 1 " could not be contacted.
          We will try contacting " nurse 2. <
    after confirming contact with nurse 2 within 20 minutes
      has failed >
        show message to patient nurse 2 " could not be contacted.
          We will try contacting " family caregiver. <
    after confirming contact with family caregiver
      within 30 minutes has failed >
        show message to patient "Nobody of your care team could be
          reached. Please call XXXXXXXX for further assistance." <<
<
```

Using an alternative syntax for assessment of time the assess time paragraph look like this:

```
order of contacting >
  in case time is between (08:00 am ... 06:00 pm) >
    sequence of contacting is (nurse 1; nurse 2;
      family caregiver). <
  in case time is between (06:01 pm ... 07:59 am) >
    sequence of contacting is (nurse 2; nurse 1;
      family caregiver). <<
...
```

Showing and sending messages

Hippocrates provides one central logic to communicate with users (locally or remote). The only default user that Hippocrates knows is the patient. Any other addressee of a message can be defined as needed.

Discussion:
1. \addressee group\ needs to be added.

```
{show or send}::
      show|send \message type\ to \addressee\ \message text\ [>
        {{\message paragraph\}}
      <].

      {message type}::
            message

            Future version
            {picture
            video}

      {message text}::
            {{[\text\] [{{\value\|\drug\|newline}}] [\text\]}} |
            ##{{[\HTML text\] [{{\value\|\drug\|newline}}]
              [\HTML text\]}}##

      {message paragraph}::
            \message option\ |
            \share message\ |
            \message to addressee\ |
            picture is \picture path\ |
            color is \predefined color\ |

            {message option}::
                  \message type\ expires after \duration\. |
                  \message type\ expires with \event\. |
                  \message type\ expires <period of time\ after \event\. |
                  show|send \message type\ earliest \period of time\
                        after \event\.
                  after delivery [within \period of time\] has failed
                    > ... < |
                  after delivery has succeeded > {{\paragraph\}} <

            {share message}::
```

```
      share message with \addressee\ [>
        {{\message option\}}<].

{predefined color}::
      red |
      green |
      yellow

{message to addressee}::
      \message type\ to \addressee\ is \message text\ [>
        {{\message option\}}<].

after delivery [within \period of time\] has failed  >
  {{paragraph}}<
```

In this paragraph two specific actions can be defined what to do with the information if the delivery has failed.

dispose \message type\.

The information will be discarded and can only be found in the log of Hippocrates.

resend \message type\ with next contact.

Next time the same addressee receives an information the information will sent as well. The information will be sent in the order newest to oldest. This is the **default option**.

Discussion(s):
1. It needs to be defined how the exact protocol for mail delivery and using a Hippocrates ID will look like.

Rules:
1. Either **dispose** or **resend** must be used in **\paragraph**.
2. **within \period of time** can be used to limit the time for retrying to send the message. In case this option is used Hippocrates will use the full time to get confirmation that the contact channel was used successful.
3. **after delivery** can only be used if **send** is used.
4. send cannot be used for **\addressee\ patient**.

Rules:
1. A **\message paragraph** can address the same **\addressee** only once.
2. **picture is \picture path** can only be used if **\message type** is picture.
3. Each **\message type\ expires...** expression can be used only once. However **\duration** and **\event** can be used together. Whatever comes first will trigger the expiration of the message.
4. **maximum time to deliver message is \period of time**, **after delivery failure** and **after delivery success** cannot be used if the addressee is the **patient**.
5. **after delivery failure** is called after all contact information have failed or the **maximum time to deliver message** has been exceeded.

Example(s):

The following examples sends a short status report to a physician every Friday.

```
every Friday at 08:00 am >
  diagnosis >
    context >
      timeframe is between (7 days ago ... now)! <
    send message to physician
      "The inhaler has been used " count of inhaler used „in
        past 7 days". >
      after delivery has succeeded >
        show message to patient "Your weekly inhaler status has
          been sent to your physician.". <
      after delivery within 10 minutes has failed >
        show message to patient "Your weekly inhaler status could
          not be sent to your physician. It will be sent next
            time your physician gets a message.".
          resend message with next contact. <<
<
```

Raising questions

Beside sending messages to different stakeholder Hippocrates also comes with the ability to raise questions.

Discussion:
1. The definition of raising questions needs more examples that help define the required actions.

```
{ask}::
      \value question\
      \inline question\

      {value question}::
            ask \addressee\ for \value\. [>
              {{\question option\}}<]
```

Comment(s):
1. Raising a question explicitly is usually not necessary as Hippocrates automatically determines when a question needs to be raised.
2. In version 1 of Hippocrates the only **\addressee** is the **patient**.

Rules:
1. **\question options** that are already defined in **\value** will be overwritten.

```
      {inline question}::
            ask \addressee\ \text\. >
              \answer assessor\
              [{{\question option\}}]
            <

      {answer assessor}::
            assess answer >
              {{\paragraph\}}<
```

Rules:
1. As **\answer assessor** describes all possible answers **\other case** must not be used.

repeat question.

Repeats the question that has been asked. *repeat question* does not lead to a new instance for the question asked.

Rules:
1. If questions are nested, *repeat question* will repeat the questions that belongs to the *{answer paragraph}* is belongs to.

stop question.

Stop a question.

Rules:
1. Stops a question and the value is set to *unknown*.

ask the same question again.

ask the same question again leads to a new instance of an answer. *repeat question* does not lead to a new instance.

```
{question options}::
      \unit handling\ |
      \question expiration\ |
      \answer validation\ |
      \style of question\ |
      \predefined answer\
```

```
{question expiration}::
      question expires after \period of time\ [>
        {{\paragraph\}}<].
```

Rules:
1. If the question is raised within an event handler **\period of time** must not be longer than the time between the events.
2. If the **\event** or the **\period** the answer is raised within end while waiting for the answer the answer also expires.

```
{answer validations}::
      validate answer \frequency\. |
      validate answer \frequency\ >
        [before \enumerator\ validation > ... <]
        [after validation has failed    > ... <]
      <
```

Rules:

1. **\frequency** cannot be larger than **twice**. This leads to a total of three times requesting the answer.

{style of question}::

```
\direct input style\ |
\Likert style\ |
\visual analogue style\ |
\selection style\
```

{direct input style}::

```
type of question is direct input.
```

{Likert style}::

```
type of question is Likert scale.
```

Rules:

1. Options to be selected are presented to the user in the order they appear in the definition of a **\value** or the **\answer paragraph**.

{visual analogue scale}::

```
type of questions is visual analogue scale [>
  best value is \number\. [>
    text for best value is "Text".<]
  worst value is \number\. [>
    text for worst value is "Text".<]
<
```

Rules:

1. If either **best value** or **worst value** is used, both options must be used.
2. **best value** and **worst value** must be within the boundaries of valid values (for **\value**) or valid answers as defined in **\answer paragraph**.

{selection style}::

```
type of question is selection.
```

Rules:

1. Options to be selected are presented to the user in the order they appear in the definition of a **\value** or the **\answer paragraph**.

{predefined answer}::

```
predefined answer is \Pvalue\.
```

Rules:

1. *<Pvalue>* must be a valid value for **\value** or any answer defined in **\answer paragraph**.

Examples:

```
ask patient "Are you ready to measure your blood pressure" >
  assess answer >
```

```
    "Yes, I am ready" > ... <
    "No, I do not want to measure my blood pressure now" > ... <<
  question expires after 20 minutes > ... <
  validate answer once >
    before first validation > ... <
    after validation has failed > ... <
  <
<
```

Constantly listening to changes of values

In some cases it might be required to constantly listening to the "outside" world for signals (new values). This listening always happens in the background. Whenever the value is updated using the Hippocrates API it can be handled using event handlers like *after <valued> changed*.

{listen}::

```
listen for \value\. [>
  [\decision tree\ >]
    reject|accept \value\. [<]
<]
```

Rules:
1. **reject \value** and **accept \value** must be used pairwise. If either statement is used the other must be used in another block.
2. The two statements cannot be used in the same innermost paragraph.
3. The validation rules of **\value** are executed before entering **\decision tree**.

`reject \value\.`

This statement rejects the value. The value will not be stored.

`accept \value\.`

This statement accepts the value. The value will be stored and the change event is triggered.

Examples:

The following example listens to signals coming from a specific disposable device. To make sure that the right signal is captured the script filters by the MAC address. This example would be typical for an automatically generated script.

```
during plan >
  listen for inhaler use >
    assess device ID for inhaler use >
      "01:23:45:67:89:ab" > accept inhaler use. <
      in all other cases > reject inhaler use. <<<
<
```

Efficient scripting

Referencing other definitions

Hippocrates will also supports referencing definitions. This will help script authors to reference either full definitions or partial definitions. What happens is that Hippocrates uses exactly the same descriptions that are is the referenced values.

The sentences used for referencing other definitions are:

```
{reference sentence}::
     definition is the same as for \value\.

{reference sentence with exception}::
     definition is the same as for \value\ except >
       {{\changed paragraph\}}
     ).
```

Rules:
1. References can only be used within a **\paragraph**.
2. Exceptions can only be defined for full **\paragraph**.
3. A **\reference sentence** can be used within an a **\reference sentence with exception**.

Example:

Let's take an example where pain levels at different points of the body should be recorded. While the position changes, the definition of pain is the same for each of the pain points.

```
"pain in left arm" is a number (
  valid value >
    criteria >
      value is 0.0?
      value is between(1.0 ... 10.0)?
    assess number of applying criteria >
      0 .. 1 > no action. <
      2      > value is valid. <<
  <

  references >
    human > somatic symptoms <
    human > body > left arm. <<
  <

  meaning >
    assess value >
               0.0 > value is not applicable.
                     meaning is "not applicable". <
      1.0 ...  2.0. > meaning is "no pain". best value. <
      2.1 ...  8.0. > meaning is "moderate pain". <
      8.1 ... 10.0. > meaning is "severe pain". worst value. <<
  <

  question >
    ask patient "What is the level of pain in your left arm".
  <
<
```

Now, we would also like to know pain the right arm and both legs. To define the value that capture these pain levels referencing if used.

```
"pain in right arm" is a number >
  definition is the same as for pain in left arm except >
    references >
      human. > somatic symptoms. <
      human. > body. > right arm. <<
    <
    question >
      ask patient "What is the level of pain
        in your right arm". <
  <
<
```

Example:

If the difference between two value is too big, but some definition should be used, partial referencing can be used.

```
"any value" is a value. >
  valid units >
    assess unit > °C; °F > unit is valid. <<<
  ...
<

"other value" is a value >
  valid units >
    definition is the same as for any value. <
  ...
<
```

Providing translations

Any script that is written in Hippocrates can be complemented with translations. Translations are a separate **Hippocrates paragraph**\ in a script.

Discussion:
1. A message could also contain values. In this case the whole message block including the value has to be provided. This would also enable the possibility to choose the right unit depending on the target language and or country.
2. In addition to point 1 it might be necessary that a geographic context can be established as this might trigger even more variants.
3. Currently only western languages are specified. It needs to be decided which Asian languages should be included. It needs to be decided which non-Latin languages should be included.

```
{Hippocrates translation paragraph}::
      translations. >
        languages. (\supported languages\).
        {{\translated text\}}
      ).
```

```
{specified languages}::
      languages (\language\; \language\[{{; \language\}}]).

      {language}::
            English
            German
            French
            Spanish
            Japanese
            Simplified Chinese
            Russian
            Hebrew

{translated text}::
      (\text in language 1\; \text in language 2\...).

      {text in language 1}::
      {text in language 2}::
            \message text\
```

Rules:
1. Translations must be provided in **\translated text** the same order as the languages are specified in **languages**.
2. Messages that are translated must be provided including any **\value** used in the source language and the target language(s).

Example(s):

At any position within a Hippocrates plan a translation paragraph can be included.

```
translations >
  languages (English; German).
  ("This is an English text"; "Das ist ein deutscher Text").
<
```

Also more complex translations can be provided. The following examples shows the translation of an information that also contains values. In addition to the text the name of the value also needs to be translated.

```
...
send message to physician
  "The inhaler has been used " count of inhaler used „in
    past 7 days".
...

translations >
  languages (English; German).
  ("inhaler used"; "Inhalator verwendet").
  ("The inhaler has been used " count of inhaler used „in
    past 7 days"; "Der Inhalator wurde in den letzten 7 Tagen "
    Anzahl von Inhalator verwendet " verwendet.").
<
```

Frequency

Frequency is used to generate repetitive events.

```
{frequency per time unit}::
      \frequency\ per \time unit\ |
      every \time unit\ |
      every \positive integer number > 1\ \time unit\
```

Rules:

1. If **\frequency\ per \time unit** is used **\frequency** must not be higher as the next smallest **\time unit**. The means that the maximum frequencies are:
   a. `once per second`
   b. `60 times per minute`
   c. `60 times per hour`
   d. `24 times per day`
   e. `7 times per week`
   f. `31 times per month`
   g. `366 times per year`

   For months and years the numbers will be corrected according to actual length of the month in the respective year or according to leap years.

~~Documentation~~

~~Hippocrates does not allow comments as all elements of the script should have an actual meaning. However, there is a possibility to add comments at any position in the script. As one core concept of Hippocrates is local language support that language context of comment must be provided.~~

```
{documentation}::
      documentation (
        {{<language> (
            <text>
        ).}}
      ).
```

~~Examples:~~

~~A comment can be inserted at any position in a Hippocrates script. Ideally the comment is so meaningful that it could also be shown to the „outside“ world.~~

```
      "Riamet 20/120mg tablets" is a drug (
        documentation (
            english (
                Here a documentation of the drug should go.
            ).
            german (
                Hier sollte die Beschreibung des Medikamentes stehen.
            ).
        ).
        valid units (
            asses unit (
                tablet (unit is valid.).
            ).
```

```
  ).

  dosage safety (
    maximum single dose = 4 tablets.
    minimum time between single doses = 8 hours.
  ).

  single dose calculation (
    assess pregnancy (
      pregnant (
        single dose = 0.
      ).
      not pregnant (
        assess weight (
          0.5 kg ...   4.9 kg (single dose = 0 tablets.).
          5.0 kg ...  14.9 kg (single dose = 1 tablet.)
         15.0 kg ...  24.9 kg (single dose = 2 tablets.)
         25.0 kg ...  34.9 kg (single dose = 3 tablets.)
         35.0 kg ... 250.0 kg (single dose = 4 tablets.)
        ).
      ).
    ).
  ).

  administration schedule (
    first dose trigger is "first dose administrated".
     8 hours after first dose administration.
    24 hours after first dose administration.
    36 hours after first dose administration.
    48 hours after first dose administration.
    60 hours after first dose administration.
  ).
).
```

Generic layout of a Hippocrates Script File

In a Hippocrates Scrip File all necessary information are put together. A Hippocrates has the following content:

```
{Hippocrates file}::
      [\intended use chapter\]
      [\library reference chapter\]
      [\settings chapter\]
      [\sharing chapter\]
      [{{\addressee chapter\}}]
      [{{\value definition chapter\}}]
      [{{\custom event definition chapter\}}]
      [{{\period definition chapter\}}]
      \plan chapter\
```

Intended use

Hippocrates requires the definition of the intended use of a script. This provides the possibility to request signing of these scripts by registered persons. This is mainly intended to help the recipient of a script to make a decision on use.

```
{intended use}::
      intended use >
        \intended use description\
        {{\intended use options\}}
      ).


      {intended use description}::
            short description is \text\.
            [long description is \text\.]


      {intended use options}::
            intended use is medical.
            intended use is non-medical.

            intended use is clinical trial.
            intended use is diagnosis.
            intended use is treatment.
            intended use is follow-up.
            intended use is behavioral change.
            intended use is ...
```

Sharing

In addition to sending information using **send message** Hippocrates provides means to send the actual raw data to any **\addressee**. In any case Hippocrates request consent to share the data. This consent either be given for all values that are requested on a per **\addressee** basis.

```
{sharing chapter}::
      sharing. >
        share with \addressee\|\addressee group\
          {{; \addressee\|\addressee group\}} >
          \event\ >
            {{\share paragraph\}} <<
      <


      {share paragraph}::
            share \shared data\. |
            share \shared data\ > |
            before consent is requested from patient > |
            after consent has been rejected by patient > |

            share \shared data\ [>
              [after delivery failure > ... <]
              [after delivery success > ... <]
            <]
```

```
{share data}::
      all values of this plan |
      all values of all plans [of same author] |
      \value\|\value group\|\drug\
      (\value\|\value group\|\drug\;
        \value\|\value group\|\drug\[{{;
        \value\|\value group\|\drug\}}])
```

Rules:
1. Each statement of **{share paragraph}** can only be used once.
2. **share \share data** must be used.
3. **\share data** cannot have the same data twice.
4. **all values of all plans** requires use of **context** with a **timeframe** expression.

Examples:

The following example shares all data that is available for a patient with an addressee (which could be a clinical data manager or even platforms like PatientsLikeMe). The sharing will only be possible, if the patient or caregiver has given the consent to share these information. This needs to be set using the Hippocrates UI. For that purpose Hippocrates will screen all values that are supposed to share and will ask fro consent to share these values. The values themselves will be packaged in JSON-format.

```
sharing >
  share with clinical data manager >
    every Friday at 8:00 pm >
      context >
        timeframe is between (7 days ago ... now)! <
      share all values of all plans >
        after delivery failure >
          show message to patient "The weekly data summery
            could not sent to the clinical data manager.
            We will get in contact with you shortly to
            assess the problem.".
          send message to service provider
            "We do not get data from trial participant XYZ.
            Please get in touch with that person within next
            2 hours.". >
              after delivery failure. >
                show message to patient "We cannot automatically
                  inform our service provider to assess the data
                  upload problem. Please call XXX-XXX-XXXXX.". <
              after delivery success. >
                show message to patient "We contacted our service
                  provider. If you do not receive a call within
                  next 2 hours, please call XXX-XXX-XXXXX.". <<<<<<
<
```

Scripting objects

Todo(s):

Period

Definition of periods

```
{period}::
      \predefined period\
      \dynamic period\

      {predefined period}::
            \name of period\ is a period. >
              \predefined period timeframe paragraph\
              [\period constraint paragraph\]
              [\period customization paragraph\]
            <

            {name of period}::
                  \text\

            {predefined period timeline}::
                  timeframe >
                    \period timeline\.
                    [{{\period timeline\.}}]
                  ).

                  Rules:
                  1. \period timeline\ cannot contain any reference to \period\ or \event\.

            {period constraint paragraph}::
                  constraints >
                    {{\period constraint\}}
                  <

            {period customization paragraph}::
                  customization >
                    {{\period customization option\}}
                  <

                  {period customization option}::
                        \begin of period customization\. |
                        duration of \period\ can be customized
                              \level of detail\.

                        {level of detail}::
                              together for all occurrences
                              individually for each occurrences

                        {begin of period customization}::
                              begin of \period\ can be customized
                                \level of detail\. [<
                                latest point in time to change begin of
                                  \period\ is \period of time\
```

```
before begin of \period\. <]
```

Rules:
1. **latest point in time to...** restricts the timeframe for changing the begin of a period. This can be used if events refer to a time prior the beginning of the period. These events should still all within the restriction timeframe, to make sure the script can be followed as outlines. This restriction is handled by the Hippocrates API by not exposing these points in time anymore.

Comment(s):
1. **\level of detail** has impact on what data is exposed by the Hippocrates core via the API. In case **for all occurrences together** is used only one data set will be created. If **for each occurrence individually** is specified the Hippocrates core will generate data sets within the forecasting horizon.

```
{dynamic period}::
      [\name of period\] \period timeline\ >
        [\period constraint\]
        {{\paragraph\}}
      <

{period constraint}::
      duration of \period\ cannot be changed. |
      \minimum duration of period\ |
      \maximum duration of period\ |
      \period\ cannot overlap with \period exclusion\. |
      \period\ must overlap with \period inclusion\.
```

Rules:
1. **duration of \period\ cannot be changed** is excluding use minimum and maximum constraints and would prevent use of any method to change the duration of a period.

```
{period exclusion}::
{period exclusion}::
      \period\
      (\period\; \period\{{; \period\}})

{minimum duration of period}::
      in case \period\ would get shorter than \period of time\
        > ... <
```

Rules:
1. If a **\minimum duration of period** is defined **\period** will never decreased to less than **\period of time**.
2. **\period of time** for **\minimum duration of period** must be less than the time specified in the definition of **\period**.

```
{maximum duration of period}::
      in case \period\ would get longer than \period of time\
        > ... <
```

Rules:
1. If a **\maximum duration of period** is defined **\period** will never increased to more than **\period of time**.
2. **\period of time** for **\maximum duration of period** must be longer than the time specified in the definition of **\period**.

Examples for predefined periods:

```
"work week" is a period >
  timeframe >
    between (Monday ... Friday). <
<

"summer" is a period >
  timeframe >
    between (June ... August). <
<

"breakfast" is a period >
  timeframe >
    between (06:15 ... 06:45). <
<
```

A period can also have multiple definitions. The expressions using days are treated as constraint whereas the expression using time of day is treated as actual duration.

```
"breakfast" is a period >
  timeframe >
    between (Monday ... Friday; 06:15 ... 06:45).
    between (Saturday ... Sunday; 08:00 ... 09:00). <
<
```

Another example is a series of periods that only take place between Monday and Friday.

```
"interval training" is a period >
  timeframe >
    between (Monday ... Friday; 10:00 ... 10:10).
    between (Monday ... Friday; 10:30 ... 10:40).
    between (Monday ... Friday; 11:00 ... 11:10).
    between (Monday ... Friday; 11:30 ... 11:40). <
<
```

The last example assumes that the script has been generated out of an EHR system and includes appointments for treatments, which might require some information to the patient before they show up for treatment. As we want to make sure that in case of date change the patient could do that himself, the beginning of the period can be changed (of cause only if this change is in the future).

```
"treatment" is a period >
  timeframe >
    between (14-Apr-2014 at 9:00 ... 14-Apr-2014 at 10:00).
    between (13-May-2014 at 11:00 ... 13-May-2014 at 12:00).
```

```
    between (11-Jun-2014 at 8:00 ... 11-Jun-2014 at 9:00).
    between (25-Jul-2014 at 18:00 ... 25-Jul-2014 at 19:00). <
  customization >
    begin of treatment can be customized for each occurrence >
      latest point in time to change begin of treatment is 2 days
        before begin of treatment. <<
    duration of treatment can be customized >
      in case duration of treatment would get shorter than
        45 minutes >
          show message to patient "The treatment takes at least
            45 minutes.". <<<
<
```

Used in a script the simple reference could look like.

```
"plan for treatment" is a plan >
  ...
  1 hour before treatment >
    show message to patient "Please take XYZ now. Your next
      treatment starts in 1 hour.".
  ).
  ...
).
```

Examples for dynamic periods:

A simple plan that runs for 9 days. If no reference to this period is required the naming ("antibiotics") can be skipped.

```
plan. >
  "antibiotics" for 9 days > ... <
<
```

To support the success of a surgery a no-smoke reminder could be issued to patients 2 weeks in advance of a planned surgery (surgery is an event).

```
plan. >
  "no smoke period"
    between (2 weeks before surgery ... begin of surgery) >
  <
  ...
  "surgery" on 30/Apr/2015.
  ...
  after surgery for 2 week > ... <
<
```

Readable period properties

Periods come with a set of properties that help. The statements that allow access to certain properties, which are:

{readable period properties}::

```
<value reference> duration of <period>
current duration of <period>
remaining duration of <period>
lapsed duration of <period>
time until next occurrence of <period>
time since last occurrence of <period>
number of <time direction> repetitions of <period>
```

{value reference} duration of <period>

Returns the *{period of time}* according to *{value reference}*, which can either be the *initial* value or the *current* duration of *<period>*.

Rules:
1. *<period>* cannot have multiple instances with different durations.
2. *<period>* must be a dynamic period created in a script.

current duration of <period>

Returns a *{period of time}* for the current instance of *<period>*. This value also reflects all changes that are made during execution of the script.

Rules:
1. The value always refers to the current instance of *<period>*.and returns the time the has elapsed since the start.
2. *current duration* returns 0 if the value is accessed outside the time boundaries of the period.

remaining duration of <period>

Returns a *{period of time}* with the remaining duration of the current instance of *<period>*.

Rules:
1. The value always refers to the current instance of *<period>*.and returns the time until the end of *<period>*.
2. *current duration* returns 0 if the value is accessed outside the time boundaries of the period.

lapsed duration of <period>

Returns a *{period of time}* with the lapsed duration of the current instance.

Rules:
1. The value always refers to the current instance of *<period>*.and returns the time until the end of *<period>*.
2. *current duration* returns 0 if the value is accessed outside the time boundaries of the period.

time until next occurrence of <period>

Returns a *{period of time}* until the next instance of *<period>* will begin.

Rules:
1. If *time until next occurrence of <period>* is used while an instance of the period is active the time until the next instance occurs will be returned.
2. If *<period>* does not occur any more in the future *unknown* will be returned.

time since last occurrence of <period>

Returns a *{period of time}* since the last instance of *<period>* has ended.

Rules:
1. If *time since last occurrence of <period>* is used while an instance of the period is active the time since the last instance has occurred will be returned.
2. If *<period>* has not occur in the past *unknown* will be returned.

number of {timeline direction} repetitions of <period>

Counts the number of occurrences of *<period>.*

Rules:
1. *past* counts the instances of *<period>* that happened in the past. If used while an instance of *<period>* is active this instance will also be counted. If there was no instance of *<period>* *none* will be returned.
2. *future* counts the instances of *<period>* that will happen in the future If used while an instance of *<period>* is active this instance will not be counted. If there will be no further instance of *<period>* *none* will be returned.

Modification of dynamic periods

Discussion(s):
1. This sections requires review

Dynamic periods can be modified in a plan.

```
{period modifications commands}::
      pause <period>
      continue <period>
      stop <period>
      <extend period>
      <shorten period>
```

pause <period>

Pause the specified period.

Rules:
1. If *<period>* is running it will be paused. If a user interaction is in place within that period the UI will be notified and no feedback to questions will be accepted any more.
2. If *<period>* is already ended, nothing will happen.
3. If *<period>* is not already running it will not start running.

4. if *pause* is used there must be an accessible block that contains a *continue* statement for the same *<period>*.

Comment(s):
1. If a *<period>* needs to be stopped finally *stop* must be used.

continue <period>

Continue the specified period.

Rules:
1. If *<period>* is not running it will be continued.
2. If *<period>* is already ended, nothing will happen.
3. If *<period>* is not already running it will start running at the specified point in time.
4. if *continue* is used there must be an accessible block that contains a *pause* statement for the same *<period>*.

stop <period>

Stop the specified period. A period that is stopped cannot be restarted.

Rules:
1. If *<period>* is not running and would be running in the future it well never start.
2. If *<period>* is already ended, nothing will happen.
3. If *<period>* is running it will immediately be ended and the UI will be notified and no answers will be accepted anymore.

{extend period}::

```
extend duration of <period> to <period of time>
extend <period relation> duration of <period> by <period of time>
```

{period relation}::

```
initial
current
```

Rules:
1. *initial* always refers to the original duration of the period.
2. *current* refers to the current length of the period reflecting the changes that might have been made already.

TODO:
1. Describe how the changes are documented in the script by using `(orginally ...)`.

Rules:
1. Whenever either *extend duration of <period> to {period of time}* or *extend duration of <period> by {period of time}* are used *maximum duration of <period> is {period of time}* must be defined.
2. A duration *<period>* can only be extended if an end is defined.
3. Periods that only have a start defined cannot be extended.
4. If a *<period>* has already ended and the extended period also ends before the current time and date no value will be changed.
5. If the value given for *{period of time}* would end the period in the past the value is adjusted to end the period *now*.

6. If the value given for *{period of time}* would lead to a period longer than the *maximum duration of <period>* the maximum duration for the period is be used.
7. The value of *{period of time}* must be greater than the value used in the original definition of *<period>*.

{extend period}::

```
shorten duration of <period> to <period of time>
shorten duration of <period relation> <period> by <period of time>
```

Rules:
1. Whenever either *shorten duration of <period> to {period of time}* or *shorten duration of <period> by {period of time}* are used *minimum duration of <period> is {period of time}* must be defined.
2. Periods can only be shortened if an end is defined.
3. Periods that only have a start point in time defined cannot be shortened.
4. A period that is running can only be shortened to the current point in time (correction happens automatically).
5. *{period of time}* defined in *shorten {period relation} <period> by {period of time}* leads to a duration that is shorter than the minimum length of the period the defined minimum duration for the period will be used.
6. The value of *{period of time}* must be less than the value used in the original definition of *<period>.*

Example(s):

A plan for an intensive treatment that can be stopped is symptoms are lower.

```
"any treatment" is a plan (
  ...
  "intensive treatment" is for 10 days (
    minimum duration of intensive treatment is 5 days.
    maximum duration of intensive treatment is 15 days (
      assess number of violations (
       all ( ... )
      ).
    ).
    ...
    3 days after begin of intensive treatment (
      every second day (
        assess treated condition (
          worse (
            increase current duration of
              intensive treatment by 1 day.
          ).
          sightly better (
            extend current duration of intensive
              treatment to 8 days.
          ).
          stable (
            stop intensive treatment.
          ).
          in all other cases (
            no –action.
          ).
```

```
            ).
          ).
        ).
      ).

      "after care treatment" after end of
        intensive treatment for 20 days (
        ...
      ).
    ).
```

Events

Events are singular points in time that cause some action or can be used as reference points.

```
{event}::
      [\catch event\] \trigger event\

{catch event}::
      with begin|end of \period\ |
      [\period of time\] before|after begin of \period\ |
      [\period of time\] before|after end of \period\ |
      [\period of time\] before|after \event\

{trigger event}::
      \repeated event\
      \time related event\

      {repeated event}::
            \event name\ \frequency per time unit\ [>
              [{{\event frequency constraint\}}]
              {{\paragraph\}}
            <].

            {event frequency constraints}::
                  \event minimum frequency constraint\ |
                  \event maximum frequency constraint\ |
                  frequency of \event\ cannot be changed.
```

Rules:
1. **frequency of <event> cannot be changed** is excluding use of any other constraint and would prevent use of any method to change the frequency of an event.

```
                  {event minimum frequency constrain}::
                        in case frequency would get lower than
                          \frequency per time unit\ > ... <

                  {event maximum frequency constraint}::
                        in case frequency would get higher than
                          \frequency per time unit\ > ... <
```

```
{time related event}::
      \event name\ \point in time\ [> ... <]
```

Example for repeated events:

```
"rehabilitation" after surgery once per day > ... <
```

Example for time related events:

```
"surgery" on 30/Apr/2015 at 10:00.
```

Readable event properties

{readable event properties}::

<value reference> frequency of <event>
time until next occurrence of <event>
time since last occurrence of <event>
number of <timeline direction> repetitions of <event>

<value reference> frequency of <event>

Returns either the *initial* or the *current* frequency of *<event>*.

time until next occurrence of <event>

Returns the time between *now* the time *{event}* will be triggered next time:

Rules:
1. If *<event>* does not happen again in the future *unknown* will be returned.

time since last occurrence of <event>

Returns the time between *now* and the last time *<event>* was triggered.

Rules:
1. If *<event>* has not happen in the past *unknown* will be returned.

number of {timeline direction} repetitions of <event>

Counts the number of occurrences of *<event>* in the specified direction of the timeline.

Rules:
1. *past* counts the instances of *{event}* that happened in the past. If *{event}* never happened in the past *none* will be returned.
2. *future* counts the instances of {period} that will happen in the future. If there will be no further instance of *{period}* *none* will be returned. If {event} relates to change events of values *unknown* will be returned.

Predefined overlap handling

If an action of an event is not completed (e.g. still waiting for questions to be answered) until the next time the event is triggered corrective action can be defined.

```
{event overlap handling}::
      in case \event\ is not finished when next \event\ occurs >
        {{[\decision\ <]}}
          \event overlap action\ {{[<]}}
      <

      {event overlap action}::
            skip new event. |
            queue new event. |
            abort outstanding actions.
```

Rules:

1. **\event overlap action** must be used as innermost statement.

Example(s):

Skip asking for the daily health status if the next one is already due. However, for the example below, the same could be achieved if the question expires after a certain period of time.

```
"assessment of health status" every day >
  in case assessment of health status is not finished when next
    assessment of health status occurs >
      abort outstanding actions. <
  ask for health status.
<
```

Skip sending the next dose reminder until the patient has answered a question about his blood pressure. In this case the next adjustment will be made when the question is answered.

```
every week "assessment of new dose" >

  in case assessment of new dose is not finished until next
    assessment of new dose >
      abort outstanding actions. <

  context is blood pressure >
    assess systolic blood pressure >
      20 mm/Hg ...  89 mm/Hg > ... <
      90 mm/Hg ... 119 mm/Hg > ... <
      ...
```

Modification of events

Events can be modified in a script to adapt their occurrence.

{modification of events}::

```
{pause event}
continue <event>
stop <event>
change frequency of <event> to {frequency} per {time unit}
decrease {value reference} frequency of <event> by
      {frequency change}
increase {value reference} frequency of <event> by
      {frequency change}
```

{pause event}::

```
pause <event>
pause<event> for [duration]
```

Rules:
1. if pause is used there must be an expression that continues the same *<event>*.
2. An*<event>* will be automatically paused if it is with a period that is paused.
3. If the *pause* statement is within the block that handles the *<event>* the block of the event will be processed to the end.
4. If *{duration}* is longer than the remaining duration of the period that event is embedded, and in the period is extended the event will not happen until *{duration}* is expired (if this could happen the Hippocrates logic analyzer should issue a warning).

continue <event>

Rules:
1. If *continue* is used there must be an expression that pauses the same<event>.

stop <event>

Rules:
1. If the *stop* statement is within the block that handles the<event> the block of the event will be processed to the end.

change frequency of <event> to {frequency} per {time unit}

Rules:
1. If *change frequency* is used *minimum frequency* and *maximum frequency* for *<event>* must be defined,
2. if *{frequency}* is less than *minimum frequency* then *minimum frequency* is used. If this issue can be identified at compile time an error will be issued and the script will not compile.3. if *{frequency}* is more than *maximum frequency* then *maximum frequency* is used. If this issue can be identified at compile time an error will be issued and the script will not compile.

decrease {value reference} frequency of <event> by {frequency change}

{frequency change}::

```
      {percentage100}
      {positive integer number} {time unit}
```

Rules:
1. If *decrease {value reference} frequency* is used *minimum frequency* for *<event>* must be defined,
2. If the change leads to a frequency less than the defined minimum frequency the defined minimum frequency is used. If this violation of the constraint can be detected to compile time the script will not compile.

decrease {value reference} frequency of <event> by {frequency change}

Rules:
1. If *increase {value reference} frequency* is used *maximum frequency* for *<event>* must be defined,
2. If the change leads to a frequency higher than the defined maximum frequency the defined maximum frequency is used. If this violation of the constraint can be detected to compile time the script will not compile.

Definition of custom events

Hippocrates also supports the definition of custom events. The feature of definition of semantic meanings can be used define any expression to require, trigger and catch events. This definition must be placed outside the plan paragraph in a script.

```
{custom event}::
    \name of event\ is an event >
      \request phrase\
      \trigger phrase\
      \catch phrase\
    ).

    {request phrase}::
          request event is \text\.
```

Defines the sentence that is used to request use of trigger phrase.

```
    {trigger phrase}::
          trigger event is \text\.
```

Defines the sentence that is actually triggering the event.

```
    {catch phrase}::
          catch event is \text\.
```

Defines the phrase that finally catches the event. The phrase must be used in combination with after (**after \catch phrase**).

Example:

The following example shows the use of custom events for a first dose administration.

```
 "first dose" is an event >
   request event is "drug requires first dose
```

```
    administration".
  trigger event is "first dose has been administrated".
  catch event is "first dose has been administrated".
<
```

Later use us of this event looks like.

```
plan >
  ...
  drug requires first dose administration.
  ...
  with begin of plan >
    ask "Did you take your first dose" >
      "Yes" > first dose has been administrated. <
      "No"  > repeat question. <<
  <
  after first dose has been administrated >
    ...
  ).
).
```

Combining periods and event in one sentence

```
<catch event>
<trigger event <repeated event>>
<period>

plan >
  ...
  "no smoke period" 2 weeks before surgery for 2 weeks > ... <
  ...
  "surgery" on 30/Apr/2015 10:00.
  ...
  "rehabilitation" after surgery for 2 week once per day > ... <
  ...
<
```

Events are the main concept of Hippocrates to trigger actions. Events can trigger actions which can either be questions, messages or response to change of values. Events can be used in plans, values and drugs.

Values

Discussion(s):
1. Is index based access necessary?
2.

The third core element of Hippocrates is values. Value can be used to store any information that is used in Hippocrates scripts. Compared to traditional languages Hippocrates stores additional information with every value.

Index based access of values

```
{index based value access}::
      \value\(\value index\)

            {value index}::
                  previous |
                  \negative integer number\
```

(Readable) value properties

Each {value} comes with some properties that can only be read in a Hippocrates script. These properties are set when a value is set via the Hippocrates API. **confidence of value**, **meaning of value** and **score of value** are writeable in the appropriate definition paragraphs.

```
{readable value properties}::
      source of value|\value\.
      name of device [of \value\].
      ID of device [of \value\]
      age of value|\value\.
      confidence of value|\value\.
      meaning of value|\value\.
      score of value|\value\.


      source of value|\value\ >> (complete assessment)
            manual | device | static assignment | dynamic assignment |
            calculation
```

Returns the source of the value.

```
      name of device [of \value\] >>
            \name of device\ | unknown
```

Returns the text description of the device that was used to generate the value. It is simply a text. This text can be compared inside the the script against e.g. validated device the value should be coming from. This validation can be placed in the validation section of a value.

<u>Example(s):</u>

In the following example some very specific validation criteria are added in the validation section of a value. Only if these criteria apply Hippocrates would accept the data that is provided via an API call. This functionality could be used in clinical trial setting where the source of data should be limited to validated / accepted sources.

```
"body weight" is a number >

  valid units >
    assess unit >
      kg; lb > unit is valid. <<<
```

```
        valid values >
          criteria >
            value is between (0.5 kg ... 150.0 kg)?
            source of value is device?
            name of device for value is "Withings scale"? <
          assess number of applying criteria >
            0 ... 2 > value is not valid. <
            3       > value is valid. <<<
      <


ID of device [of \value\] >>
      \MAC address of device\ | unknown
```

Returns the text description of the device ID that was used to generate the value. It is simply a text. This text can be compared inside the the script against e.g. validated device the value should be coming from. This validation can be placed in the validation section of a value.

Example(s):

If one extends the example from above using ID of device could further increase the specificity of the data source. In this case only data from a specific device would be accepted. These type of scripts could be automatically generated by a clinical trial enrollment tool.

```
      "body weight" is a number >
        valid units >
          assess unit >
            kg; lb > unit is valid. <<<

        valid values >
          criteria >
            value is between (0.5 kg ... 150.0 kg)?
            source of value is device?
            ID of device for value is "01:23:45:67:89:ab"? <
          assess number of applying criteria. >
            0 ... 2 > value is not valid. <
            3       > value is valid. <<<
      <
```

Definition of values

```
{value}::
      \name of value\ is \type of value\ (
        {{\value paragraph\}}
      ).

      {name of value}::
            \text\

      {type of value}::
            number
```

```
        enumeration
        time indication
        (picture)  // Required for version 1?
        (video)    // Required for version 1?

{value paragraphs}::
        \intended use paragraph\ |
        \time paragraph\ |
        \valid unit paragraph\ |
        \valid values paragraph\ |
        (\question paragraph\ | \calculation paragraph\) |
        \confidence paragraph\ |
        \reuse paragraph\ |
        (\meaning paragraph\ | \score paragraph\) |
        \presentation paragraph\ |
        \references paragraph\ |
        \sharing paragraph\ |
        \value updates paragraph\
```

Intended use (required)

Intended use helps to define constraints on how the value can be used in scripts. Within this paragraph it can be controlled what actions can be taken with a value.

Caution:
1. Changing any intended use settings might invalidate parts or all of the scripts if the intended use rules become violated.

```
{intended use paragraph}::
      intended use (
        {{\intended use definition\}}
      ).

      {intended use definition}::
            for use in value groups only. |
            for use as constant only. |
            represents a concentration. |
            timeframe of value is \period of time\.

            represents a concentration.
```

Concentrations must be handled carefully as there must be explicit factors defined to convert between commonly used units mmol/L or mg/dL. If this statement is present Hippocrates request a conversion rule between all units used in a script.

```
            timeframe of value is \period of time\.
```

This statement describes the timeframe the value is referring to. A simple example would be steps per day. This statement is especially useful if the value should be extracted from data sources like Apple HealthKit or Google Fit.

Example:

```
"steps per day" is a number >
  intended use >
    timeframe of value is 1 day. <

  valid values > ... <
<
```

Valid units

Hippocrates supports definition of units for numbers and time related values. If units are defined they have to be used in the script next to the value.

```
{valid unit paragraph}::
      value units >
        \unit definition\ <
```

Example(s):

Values that are defined with numbers must be referred to with numbers.

```
"height" is a number >
  valid units >
    assess unit >
      m; cm; inch; foot > unit is valid. <<
  valid values >
    assess value >
      0.4 m ... 2.5 m > value is valid. <<<
  ...
<

...height is between (0.4 m ... 1.0 m)...
```

Units can also be a bit more "exotic" to support better readability of the script. In the following case the unit is defined as *steps in 6 minutes*.

```
"walk test" is a number >
  valid units >
    assess unit >
      "steps per 6 minutes" > unit is valid. <<<
  valid values >
    assess value >
      0 steps per 6 minutes ... 10000 steps per 6 minutes >
        value is valid. <<<
<
```

{valid values paragraph}::

```
      valid values >
        assess value >
          {{\range selector\ > value is valid. <}}<
```

```
< |

valid value >
  criteria >
    {{\assessment question\}} <
  assess number of applying criteria >
    {{\range selector\ > \validity of value\ <}} <<
```

{assessment question}::

```
      \readable value property\ is \criteria\?
```

Rules:
1. **\assessment questions** can refer to any **\readable value property**.
2. Using **age of value** also allows explicitly setting the timestamp of a value via the Hippocrates API. The criteria used must be between some point in time in the past (**\period of time\ ago**) and **now**.

{validity of value}::

```
      value is valid. |
      value is not valid.
```

Rules:
1. **value is valid** must be used at least once.
2. If **\value** is a number or a time ranges (either based on **\range selector** or **between** expressions) must not overlap or be completely within the range of another specified range.
3. In case more than one unit is defined for **\value** can be used and conversion rules for units are defined the same unit must be used in all validation expressions.
4. In case no conversion for units are defined the keyword **unit|units** replaces the actual unit.
5. In case **value** is a number all numbers used in validation expression must have the same precision. This precision is also used for input precision (see Hippocrates API).
6. If a validation rules contains a reference to a **\value** all values must be part of a **\value group**.
7. If an **\assessment questions** is based on other values these values must be within the value boundaries of **\value**.

Example(s):

The following example shows the validation of **systolic blood pressure** which has two conditions. One is the general range and another one is the dependency to **diastolic blood pressure** which must be lower. Please note, that this dependency is only possible if **systolic blood pressure** and **diastolic blood pressure** are part of a value group (in this case **blood pressure**).

```
"systolic blood pressure" is a number >
  intended use >
    for use in value group only. <
  valid units >
    assess unit >
      "mm/Hg" > unit is valid. <<<
  valid values >
    criteria >
      value is between (20 mm/Hg ... 400 mm/Hg)?
      value is between (diastolic blood pressure ... 400 mm/Hg)? <
    assess number of applying criteria >
      0 ... 1 > value is not valid. <
```

```
      2       > value is valid. <<<
<
```

The next example is an enumeration to assess pain. Here the complexity is much less than for blood pressure in the previous example.

```
"pain" is an enumeration >
  valid values >
    assess value >
      "none"; "mild"; "moderate"; "severe" > value is valid. <<<
<
```

The following examples defined two dependent values that could use different units but no conversion is provided.

```
"maximum dose" is a number >
  valid unit >
    assess unit >
      "tablet"; "drop"; "mg" > unit is valid. <<<
  valid values >
    assess value >
      1 unit ... 100 units > value is valid. <<<
  ...
<

"prescribed dose" is a number >
  valid unit >
    definition is the same as for maximum dose. <
  valid values >
    criteria >
      value is between (0 units ... 100 units)?
      value is between (0 units ... maximum dose units)? <
    assess number of applying criteria >
      0 ... 1 > value is not valid. <
      2       > value is valid. <<<
  ...
<

"treatment" is a plan >
  ...
  maximum dose is 4 tablets.
  prescribed dose is 3 tablets.
  ...
<
```

In the following example <value> is a time indication and it should only be possible to enter dates between now and sometime in the past. In this case the full possible range has to be specified in one condition and the second condition narrows down the dates that are possible.

```
"time of last seizure" is a time indication >
  valid units >
    assess unit > date > unit is valid. <<<
  valid value >
```

```
        criteria >
          value is between (01-Jan-1950 ... 01-Dec-2030)?
          value is between (01-Jan-1950 ... now)?
        assess number of applying criteria >
          0 ... 1 > value is not valid. <
          2       > value is valid. <<<
    <
```

Question

In many cases a value needs to be acquired by asking a person. This question can be embedded in the definition of a value to make the actual script more readable.

```
{question paragraph}::
    question >
      ask "Question text" [>
        \answer assessor\
        [{{\question option\}}]<]
    <
```

Rules:
1. If **\answer assessor** is not used, the possible answers will be constructed from **\valid values paragraph**.
2. If **\answer assessor** is defined the full valid range specified in **\valid values paragraph** needs to be covered.

In **\question paragraph** an addition phrase is available:

```
in case confidence of answer is \confidence range\ > ... <
for all other values of confidence of answer > ... <
```

With that sentence an action depending on the confidence of the answer can be triggered.

Rules:
1. To evaluate the confidence value **\confidence paragraph** must be defined.
2. **in case confidence of answer** can be used multiple times, as long as **\confidence range** does not overlap.
3. If used **\confidence range** ranges do not cover the range **0% ... 100%** the expression **for all other values of confidence of answer** must be used.

Example(s):

To be written (for the moment see "Raising questions").

```
"age" is a number >
  valid units >
    assess unit > year > unit is valid. <<<
  valid values >
    assess value >
      0 years ... 150 years > value is valid. <<<
  ...
```

```
  confidence >
    assess value >
        0 years ...  99 years > confidence of value = 100%. <
      100 years ... 150 years > confidence of value = 80%. <<<
  ...

  question >
    ask "What is your age in years?" >
      in case confidence of answer is between (0% ... 79%) >
        validate answer once >
          before first validation >
            show message to patient "Please confirm your age." <<<
      for all other values of confidence of value > no action. <<
<
```

Alternative style using assess confidence of value.

```
question >
  ask "What is your age in years?" >
    assess confidence of answer >
      100%       > no action. <
      0% ... 99% >
        validate value once >
          before first validation >
            show message to patient
              "Please confirm your age." <<<<<<
```

Calculation

An alternative way to acquiring a value through a question is calculating it. Hippocrates determines each time the value is accessed if the value needs to be calculated. In case other values are necessary for that calculation Hippocrates "follows" the chain of values that are needed and triggers the required actions (which could be calculation other values or raising questions).

```
{calculation paragraph}::
      calculation >
        [\decision\ >]
            value = <Rvalue>. [<]
      <
```

<u>Rules:</u>

1. **value = \Rvalue** must be the innermost expression.
2. Any control structure that is used cannot be ambiguous allowing to reach **value = \Rvalue** multiple times (e.g. having a default calculation at the top of the paragraph and overriding that in a later statement based on a decision.
3. Only valid values can be assigned. If a **\formula** is used Hippocrates will pre-evaluate the formula to determine if min and max values are within the specified range of **\value**.
4. The confidence of the value is set to the lowest confidence of any value used (weakest link in the chain) in the calculation or any decision used before reaching the assignment **value = \Rvalue**.
5. The calculation is only executed, if
    a. the value is **unknown**.
    b. the reuse period has been expired
    c. the required confidence level of the current value is not sufficient

d. any value that is used in the calculation has been changed

6. **\value** with **\calculation paragraph** cannot be used as **\Lvalue**.

Example(s):

The example shows the calculation of the body mass index (**BMI**). This is straight forward. What is more complex. In the example the it will be checked at compile time what the extreme values of the formula can be based on the input values **height** and **body weight**.

```
"BMI" is a number >
  valid units >
    assess unit >
      "kg/m^2" > unit is valid. <<<
  valid values >
    assess value >
      0 kg/m^2 ... (400 kg)/(2.5 m)^2 > value is valid. <<<
  calculation >
    value = (height in m) / (body weight in kg)^2. <
<
```

Confidence

Value in Hippocrates all comes with a confidence value that changes over time. Confidence control is intended to be used if decisions require a high confidence in the values involved in the decisions.

```
{confidence paragraph}::
      confidence >
        [\decision\ >]
          confidence of value = \percentage100\. [<]
      <
```

Rules:

1. **confidence of value = \percentage100** must be the innermost sentence.
2. **confidence of value = \percentage100** cannot be called more than once in the same path.
3. If **\confidence paragraph** is not specified a confidence of 100% is assumed.
4. The **\confidence paragraph** will also be used to assess the confidence of an answer with **\question paragraph**.
5. The actual confidence of the value is calculated as **confidence of value** minus the decline of confidence as defined in **\reuse paragraph**.
6. **\confidence paragraph** can only be used if **\calculation paragraph** is present. For **\calculation paragraph** the confidence is based on the values that are used in the calculation.

Reuse

Hippocrates can restrict the use of values to a certain period of time. If the value is used after that period Hippocrates will raise questions to acquire these values again.

```
{reuse paragraph}::
      reuse >
        [\decision\ >]
```

```
    \reuse\ [<]
<.

{reuse}::
     reuse period of value is \period of time\ [with
       \percentage100\ linear decline of confidence].
```

Rules:
1. **\reuse definition** must be the innermost sentence.
2. Any control structure that is used cannot be ambiguous allowing to reach **\reuse definition** multiple times at the same time.
3. If **\reuse paragraph** is not specified an infinite reuse period is assumed.

Example(s):

A simple example is weight where the reuse time could depend from the age.

```
"weight" is a number >
  ...
  reuse >
    assess age >
      0 years ... 1 year >
        reuse period of value is 2 weeks with 30% linear decline
          of confidence.
      2 years ... 10 years > ... <
      in all other cases >
        reuse period of value is 6 months with 20% linear decline
          of confidence. <<<
<
```

Alternative style

```
reuse >
  in case age is between (0 years ... 1 year) >
    reuse period of value is 2 weeks with 30% linear decline
      of confidence. <
  in case age is between (2 years ... 10 years) > ... <
  for all other values of age >
    reuse period of value is 6 months with 20% linear decline
      of confidence. <
<
```

Meaning

Interpretation of data is supported by meanings.

```
{value meaning paragraph}::
      meaning >
        \decision\ >
          {{\meaning\}} <
      <

      {meaning}::
```

```
meaning of value = \meaning of value\. |
value is best value. |
value is worst value. |
value is not applicable value.

{meaning of value}::
      \text\
```

`value is best value`

Specifies the best value of \value\. This information is used for statistical purposes.

`value is worst value`

Specifies the worst value of *{value}*. This information is used for statistical purposes.

`value is not applicable value`

Specifies the value of *{value}* that "represents" a not applicable value (e.g. in case the questions was not answered or "not applicable" was selected for a question. This information is used for statistical analysis.

Rules:
1. **\meaning** can only be used within a **\decision**.
2. **value is best value**, **value is worst value** and **value is not applicable value** cannot be used with **\range** decision. Each of these expressions can be used only once.
3. **value is best value** and **value is worst value** must be used together.
4. **value is best value** and **value is worst value** can only be used with numbers.
5. **value is not applicable value** must be outside the range that is created by **value is best value** and **value is worst value**.

Comments:
1. *value is best value* and *value is worst value* can currently only be used with a linear definition of a range (like questions) . With *<value>* definitions that have "normal" value that is in the middle and value ranges left and right it currently does not work.

Example(s)

A value can have many different interpretations that could depend from other values. In the following example the respiratory rate will be assessed. As the assessment is dependent on the age these distinctions will be made. Please not that Hippocrates will make the decision if questions about age need to be raised or not. When using the assessment in a script instead of using the actual numerical number the meaning of that value can be used. This increases readability and reduces potential errors.

```
"respiratory rate" is a number >
  valid units >
    in case unit is one of ("breaths per minute"; "bpm") >
      unit is valid.
      1 breaths per minute = 1 bpm.
      1 bpm = 1 breaths per minute. <
  <
```

```
  valid values >
    in case value is between (0 bpm ... 100 bpm) >
      value is valid. <
  <

  meaning >
    assess age >
      0 years >
        assess pediatric age >
          0 months ... 2 months >
            assess value >
               0 bpm ...  19 bpm >
                          meaning of value = "critically low". <
              20 bpm ...  29 bpm >
                          meaning of value = "low". <
              30 bpm ...  60 bpm >
                          meaning of value = "normal". <
              61 bpm ...  80 bpm >
                          meaning of value = "high". <
              81 bpm ... 100 bpm >
                          meaning of value = "critically high". <<<
            3 months ...
<
```

Alternative syntax:

```
meaning >
  in case age is 0 years >
    in case pediatric age is between (0 months ... 3 months) >
      assess value >
         0 bpm ...  19 bpm >
              meaning of value = "critically low". <
        20 bpm ...  29 bpm >
              meaning of value = "low". <
        30 bpm ...  60 bpm >
              meaning of value = "normal". <
        61 bpm ...  80 bpm >
              meaning of value = "high". <
        81 bpm ... 100 bpm >
              meaning of value = "critically high". <<<
    in case pediatric age is between (3 months ... 12 months) >
      ... <
  in case age is between (1 year ...
  ...
  for all other values of age >
    ... <
<
```

Instead of directly analyzing the respiratory rate later in a script the meaning of the value can be used.

```
...
assess respiratory rate >
  critically high; high > ... <
```

```
      normal > ... <
      critically low; low > ... <<
    ...
```

Scoring

For all enumerations a score can be defined. This simplifies statistical analysis of enumerations as a score can be added to each valid value.

```
{score paragraph}::
    scoring >
      \decision\ >
        {{\scoring\}} [<]
    <

    {scoring}::
        score of value = \number\. |
        score is best score. |
        score is worst score. |
        score is not applicable score.
```

score is best score

Specifies the best value of **\value**. This information is used for statistical purposes.

score is worst score

Specifies the worst value of **\value**. This information is used for statistical purposes.

score is not applicable score

Specifies the value of **\value** that "represents" a not applicable value (e.g. in case the questions was not answered or "not applicable" was selected for a question. This information is used for statistical analysis.

Rules:
1. **\scoring** can only be used within a **\decision**.
2. **score is best score**, **score is worst score** and **score is not applicable score** cannot be used with **\range** decision. Each of these expressions can be used only once.
3. **score is best score** and **score is worst score** must be used together.
4. **score is best score** and **score is worst score** can only be used with enumerations.
5. **score is not applicable score** must be outside the range that is created by **score is best score** and **score is worst score**.

References

References will help to reference a value to a common medical term. For that purpose Hippocrates includes different medical models (to be specified).

```
{references paragraph}::
      references >
        {{\reference\}}
      }

      {reference}::
            human >
              anatomy >
                head >
                neck >
                torso >
                arms >
                legs >
```

This section needs to be written by a medical person.

{sharing paragraph}::

THIS SHOULD BE REPLACED BY LIBRARY MANAGEMENT

~~*sharing of value (*~~
~~* value is shared between plans [<sharing restriction>].*~~
~~*).*~~

~~***{sharing restriction}::***~~
~~*of same author*~~

~~Rules:~~
~~1. For values to be shared both values must be named identically.~~
~~2. For a value to be used in different plans the total range of values specified in *{validation paragraph}* must be identical.~~
~~3. For a value to be used in different plans the valid units specified in *{valid unit paragraph}* must be identical. If the units in one plan are subset of the other and conversion rule are defined the values are still treated as identical.~~
~~4. If a value that is shared is requested all validation criteria in all plans must apply.~~

~~Example(s):~~

~~In one script systolic blood pressure is defined as follows.~~

~~*"systolic blood pressure" is a value (*~~
~~* valid units (*~~
~~*   valid unit is "mm/Hg".*~~
~~*).*~~
~~* validity of value (*~~
~~*   assess value (*~~
~~*   20 mm/Hg ... 400 mm/Hg (value is valid.).*~~
~~*   ).*~~
~~*).*~~
~~* ...*~~
~~*).*~~

~~In another plan systolic blood pressure is defined as well, but as it used in a clinical trial it must be measured with a specific device and has a more narrow range.~~

```
"systolic blood pressure" is a value (
  valid units (
    valid units are ("mm/Hg", "bar") (
      1 mm/Hg = 0.001333 bar.
      1 bar = 750.061683 mm/Hg.
  ).
  validity of value (
    assess (
      value is between (30 mm/Hg ... 300 mm/Hg)?;
      device id of source for value is "01:23:45:67:89:ab"?
    ) (
      assess number of applying criteria (
        2 (value is valid.).
        0 ... 1 (value is not valid.).
      ).
    ).
  ).
  ...
).
```

~~For the validation of a systolic blood pressure value that is shared between plans Hippocrates will created the following set of validation criteria. For the actual range that can be used the smaller band will be used for the assessment.~~

```
validity of value (
  assess (
    value is between (20 mm/Hg ... 400 mm/Hg)?;
    value is between (30 mm/Hg ... 300 mm(Hg)?;
    device id of source for value is "01:23:45:67:89:ab"?
  ) (
    assess number of applying criteria (
      3 (value is valid.).
      0 ... 2 (value is not valid.).
    ).
  ).
).
```

Timed update

The update of a value can be triggered times based. This functionality could be used to e.g. increase an age value automatically after one year.

```
{times update paragraph}::
      timed update >
        {{\update event\ >
          value = \Rvalue\. <
      <

      {update event}::
            every \period of time\
            \frequency\ \unit of time\
```

calculated value

Contains the value that is calculated in **\calculation paragraph**.

Rules:
1. If **calculated value** is used **\calculation paragraph** must be defined.
2. **\update** is used that time starts with the last change of the value.

Example(s):

Update an age value once a year.

```
"age" is a number >
  ...
  times update >
    every year > value = value + 1 year. <<
  ...
<
```

Value related events

Changes of value can also be used to trigger events.

{catch value events}::

```
change of <value>
```

Rules:
1. A change event is always triggered when a value is changed.
2. **\catch value events** can only be defined once in a chapter.

Example(s):

In the following example the script asks for a value, that the application Hippocrates is embedded in, will automatically provide once the signal is detected.

```
plan (

  during plan (
    listen for inhaler used.
  ).

  after change of inhaler used (
    ...
  ).

).
```

{special value accessors}::

```
meaning of <value>
```

```
score of <value>
source of <value>
name of device for <value>
ID of device for <value>
```

Value groups

Value groups are a mean of combining different single values in one logical group. An example for a logical group can be a questionnaire or an activity with a duration.

```
{value group}::
      <name of value group> is a value group (
        <grouped values>
        [<value group question paragraph>]
        [<value group meaning paragraph>]
      ).

{grouped values}::
      grouped values (
        {{<value>; <value> (<consolidation rule>}).}}
      ).


{value group question paragraph}::
      question (
        ask <question text> [(
          [<question expiration>]
          [<question progression assessor>]
        )].
      ).

      {question text}::
            <text>
```

Rules:
1. The questions will be asked in the same order as the values are specified in <grouped values>.

```
{value group meaning paragraph}::
      <meaning paragraph>
```

Rules:
1. If *<meaning paragraph>* is defined is hides all *<meaning paragraph>* definition all values in the group.

Example:

```
      "type of exercise" is an enumeration (
        valid values (
```

```
    assess value (
      "cycling"; "tennis"; "football" (value is valid.).
    ).
  ).
).

"duration of exercise" is a time indication (
  valid units (
    assess unit (
      minutes; min (unit is valid.).
    ).
  ).
  valid values (
    assess value (
      0 min ... 1440 min (value is valid.).
    ).
  ).
).
```

Now the two values are grouped and at one point in time the user will be asked which exercises he did how often during a day.

```
"daily exercise" is a value group (
  grouped values are (type of exercise; duration of exercise).

  question (
    ask "What exercises did you do for how long today?" (
      assess progression of question (
        question completed (
          ask "Did you do further exercises" (
            "yes" (ask same value group question again.).
            "no" (finished.).
          ).
        ).
      ).
    ).
  ).
).
```

Discussion(s):

1. The example above could generate a problem as a user could selected the same type of exercise multiple times in the iterations?

    Potential solutions:

    a. Introduce a mean that a series of values can be captured where certain rules have to be followed.

2. What would be the system response if the used selected Yes but would not want to enter additional exercises?

Information

Hippocrates allows to define information that cannot contain any values. These messages can be used for e.g. drug information or any other static messages that should be given to the patient. However, these information can also be sent to any other addressee.

Discussion:
1. It should be possible to include pictures in an information.

```
{information}::
      \text\ is an information >
        {{\language\ >
          \text of information\ <}}
        color is \predefined color\.
      <
```

Information group

Information groups can be used to bundle a set of information. Any information in this bundle can then be shown to an address either using the sequence that is used in the group definition or even randomized and repeated. The latter is especially useful to show

```
{information group}::
      \text\ is an information group >
        grouped information are >
          \information\{{; \information\}}. <
        order of use >
          use information in sequential order.
          use information randomly [only once].
          provide access to [already used] information.
        <
      <
```

Drug

Discussion(s):
1. This whole ares needs to be thought through. It is the first attempt and requires more examples.

Accessible values in context of <drug>

single dose
maximum single dose
minimum period between single doses
prescribed dose
actual dose

```
{drug}::
      <name of drug> is a drug (
        <valid drug unit paragraph>
        <dosage safety paragraph>
        <single dose calculation paragraph>
        <administration schedule paragraph>
      ).

      {valid drug unit paragraph}::
            <valid unit paragraph>
```

```
{dosage safety paragraph}::
     dosage (
       maximum single dose = <number> <valid unit>.
       minimum period between single doses = <period of time>.
     ).

     administration (
```

Examples:

```
"Riamet 20/120mg tablets" is a drug (
  valid units (
    asses unit (
      tablet (unit is valid.).
    ).
  ).

  dosage safety (
    maximum single dose = 4 tablets.
    minimum time between single doses = 8 hours.
  ).

  single dose calculation (
    assess pregnancy (
      pregnant (
        single dose = 0.
      ).
      not pregnant (
        assess weight (
           0.5 kg ...   4.9 kg (single dose = 0 tablets.).
           5.0 kg ...  14.9 kg (single dose = 1 tablet.)
          15.0 kg ...  24.9 kg (single dose = 2 tablets.)
          25.0 kg ...  34.9 kg (single dose = 3 tablets.)
          35.0 kg ... 250.0 kg (single dose = 4 tablets.)
        ).
      ).
    ).
  ).

  administration schedule (
    first dose trigger is "first dose administrated".
     8 hours after first dose administration.
    24 hours after first dose administration.
    36 hours after first dose administration.
    48 hours after first dose administration.
    60 hours after first dose administration.
  ).
).


plan (
  use Riamet 20/120mg tablets.

  context is Riamet 20/120mg tablets (
    with begin of plan (
      ask "Please confirm once you have take the first
```

```
        dose: " single dose (
        assess answer (
          "First dose has been taken" (
            first dose administration.
        ).
      ).
    ).

    30 minutes before administration (
      show message to patient "Your next dose is due in 30
        minutes. Please make sure that you administer Riamet
        with milky food.".
    ).

    with administration (
      show message to patient "Please take your next dose now: "
        single dose.
    ).
  ).
).


"biologics" is a drug (
  valid units (
    asses unit (
      mg (unit is valid.).
    ).
  ).
  dosage safety (
    maximum single dose = 200 mg.
    maximum dose = 400 mg.
    minimum time between administration = 6 months.
  ).
  administration schedule (
    first dose trigger is "first dose administrated".
     8 hours after first dose administration.
    24 hours after first dose administration.
    36 hours after first dose administration.
    48 hours after first dose administration.
    60 hours after first dose administration.
  ).
).
```

Things to address

Library management and versioning
Script version management
Handling script updates

Logging of what has happened

How to deal with plural and singular

- -

Continuous reading of a blood pressure value every second.

```
"device readiness" is an event (
).

blood pressure reading requires device readiness.

ask "please sit down and confirm and confirm once the device ready" (
  assess answer (
    "Yes" (
      blood pressure device is ready.
    ).
    "No" ( . . . ).
  ).
).

"5 minute blood pressure reading"
  after blood pressure device is ready for 5 minutes (
  every second (
    read blood pressure.
  ).
).

after 5 minute blood pressure reading (
  assess blood pressure on timeline (
    5 minutes ago ... now (
      assess sampling with sampling interval (
        10 seconds ( . . . ).
      ).
    ).
  ).
).
```

- -

Limited HTML control of messages

```
show message to patient ##"<p>Your dose is: " daily dose with units
  </p><p>Please make sure you take the " unit of daily dose
  " within next 5 minutes</p>".##
```

- -

```
"Riamet 20/120mg tablets" is a drug (
  administration (
    form of administration is tablets.
    maximum single dose = X tablets.
    maximum daily dose = X tablets.
    minimum time between doses = {period of time}.
```

```
  ).

  calculation (
  ).

  ingredients (
    active ingredients are ("ingredient 1"; "") (
      ingredient 1 (substance amount is XXX <unit>. metabolism...?).
      ...
    ).
  ).

  interaction (
    assess interaction with other administrated drugs (
      "ingredient it could interfere with" ( .... ).
    ).
  ).

).

{drug}::
      "drug" is a drug (
        {administration}
        [{single dose calculation}]
        [{ingredients}]
        [{interactions}]
      ).

      {administration}::
            administration (
              form of administration is {drug unit}.
              maximum single dose = X {drug unit}.
              maximum daily dose = X {drug unit}.
              minimum time between doses = {period of time}.
            ).

      {single dose calculation}::
            calculation (
              {{{sentence} (
                {{{sentence} ({}}
                  single dose = <number> {drug unit}
                {{).}}
              ).}}
            ).


      {ingredients}::
            ingredients (
              active ingredient is "ingredient". |
              active ingredients are ("ingredient"[{{; "ingredient"}}]).
              [assess ingredients (
                {{<ingredient> (
                  amount of substance = <number> mg.
                  ?Pharmacokinetic values?
                ).}}
              ).]
```

```
      ).

{interactions}::
      interactions (
        assess interaction with (
          "ingredient <drug> interacts with" (
            show message to {addressee} "Message".
            [stop plan.]
          ).
        ).
      ).
```

HealthScript
Enabling innovation in health
TimeScript
WorldScript